

VedGraph: Agentic Construction of an Evidence-Typed Knowledge Graph for the Vedic Saṃhitās

Himanshu Mohanty

Independent researcher

<https://github.com/HimanshuMohanty-Git24/VedAnvaya>

Preprint. Measurements frozen at commit 6bf220c, 20 September 2026, from a working tree carrying this paper’s own untracked directory and nothing else (Appendix H).

Abstract

Large language models can read a historical corpus far faster than a philologist can, and that speed is the problem: a model proposes fluent claims that are indistinguishable, once written into a graph, from claims a source actually makes. We report the construction of **VedGraph**, a knowledge graph over the four Vedic Saṃhitās in a single named recension each — Rgveda Śākala, Sāmaveda Kauthuma Ārcika, Vājasaneyi Mādhyandina, and Atharvaveda Śaunaka, 20,210 canonical mantras in total — built over seventeen days by a human owner directing bounded LLM agents alongside deterministic pipelines. The graph holds 165,737 nodes and 510,905 relationships.

The method rests on two commitments. First, identity is source-independent: a passage’s identifier is a version-5 UUID over a canonical URN built from recension and structural coordinates alone, and every normalisation — accent folding, script transliteration, sandhi-insensitive comparison — is a comparison instrument that may generate candidates but may never establish that two passages are the same. Second, every relationship carries the epistemic layer that produced it: source-explicit, deterministically derived, model-extracted, or interpretive. Because the layer is a stored, indexed property rather than documentation, its distribution is measurable. It is the paper’s central empirical result: although agents were used throughout construction, **0.17 % of relationships are model-extracted** (890 of 510,905), against 21.39 % source-explicit and 77.22 % deterministically derived. The ratio depends on the population and we give four; over the semantic assertion layer alone, where a model is the proposing mechanism, the share is 7.0 % — and none of it was promoted. The agentic contribution was concentrated in candidate generation, measurement, cross-file reconciliation and adversarial refutation — not in assertion. A policy expressed as data rather than as convention enforces this: a model may write exactly one lifecycle status, CANDIDATE, and the promotion gate is a frozen set of trust classes that does not contain the model’s.

We describe the validation regime that made the ratio hold, in which each layer is attacked by an independently implemented gate, and we report its failures as evidence about method. The project deleted 41,796 of its own relationships in one revision and retired 30,539 in the next; withdrew 96 % of its semantic role fillers rather than ship a measured 24.8 % error; refused an entire corpus on an inter-reader agreement of 0.505; and refused a semantic-resemblance layer after a re-implemented lexical control outperformed it. In the case we treat at greatest length, a gate built to attack the project’s own *withholdings* — not only its assertions — found 39 withheld Sāmavedic verses whose stated reason the repository’s own released data disproved; the reason was corrected and no verse moved from withheld to released. We state plainly what no amount of this machinery establishes: of 35,131 semantic assertions, zero have been reviewed by a human, and the project’s own certification says so. We argue that in scholarly knowledge-graph construction the durable contribution of agents is not the claims they generate but the refutations they can be made to perform, and that this is only safe when the graph can say, per edge, how each claim came to be.

Keywords: knowledge graph construction; LLM agents; provenance; evidence typing; adversarial validation; digital humanities; computational philology; Sanskrit; Vedic corpus; text reuse.

1 Introduction

A knowledge graph built over an ancient corpus has to answer a question that a graph built over a product catalogue does not: *who says so*. That a hymn is addressed to Indra may be a statement of the traditional index, an inference from the hymn’s own wording, a modern editor’s normalisation, or a language model’s reading. All four can be written as the same triple. Once they are, the distinction is unrecoverable, and every downstream count silently mixes them.

This paper reports the construction of **VedGraph**, a knowledge graph over the four Vedic Saṃhitās, and the method that produced it. The method’s defining feature is that large language models were used pervasively and were never permitted to assert. Agents read sources, proposed candidates, wrote queries, measured populations,

reconciled disagreeing files, and — most consequentially — attacked the project’s own output. Deterministic code and a human owner decided what became a fact. The paper’s argument is that this division is not a matter of discipline but of mechanism, and that the mechanism is worth describing because it is measurable in the artefact it produced.

1.1 The problem

Three failure modes recur when models are put to work on historical text, and all three are failures of *typing* rather than of accuracy.

Fluency passes for evidence. A model’s confidence score is produced by the same process that produced the claim; thresholding on it selects for fluency, not for truth. The project states this as policy and refuses to use confidence as an acceptance criterion at all (Section 4.3).

Normalisation passes for identity. To compare a Rigvedic verse with its Sāmavedic counterpart one must fold accents, scripts, punctuation and word division. Having folded them, it is one short step to concluding that two verses which fold together *are* the same verse. They may not be: *aśvaḥ* and *aśvāḥ* share an ASCII fold and are two words.

Absence passes for zero. A query that returns only positive rows invites the reader to infer that the unreturned rows are negative. In a corpus where a layer may be genuinely absent, deliberately withheld, or simply never assessed, an untyped absence is the most misleading thing a graph can publish.

1.2 Research questions

RQ1 How can a heterogeneous historical corpus be transformed into a large, provenance-aware knowledge graph with substantial LLM-agent involvement, *without* model-generated hypotheses becoming indistinguishable from source-explicit facts?

RQ2 What division of labour between deterministic computation, agents, and human decision actually emerges when the constraint in RQ1 is enforced by mechanism rather than by convention — and what share of the resulting graph do agents in fact author?

RQ3 Can adversarial validation by independently implemented gates raise reliability when no comprehensive human gold standard exists, and what, precisely, does it fail to establish?

1.3 Contributions

1. **A recension-explicit identity architecture** for a four-Saṃhitā corpus (Section 3). Passage identity is a UUIDv5 over a canonical URN composed of recension and structural coordinates only; no source, edition, text, script or sequence position may enter it. We verified the scheme reproduces all 22,537 shipped passages with zero key, URN or UUID collisions. A separate *referent-drift guard*, built after a segmentation change silently moved five passages under unchanged identifiers, distinguishes “the stored text changed” from “what this identifier denotes changed” — a distinction identity hashing alone cannot make.
2. **An evidence-typed graph model** (Section 4) in which four orthogonal axes travel with every edge: the epistemic layer that produced the claim, a reader-facing quality tier derived from it, the *attribution precision* separating a per-verse statement from a label inherited from its enclosing hymn from the verse’s own wording, and the *textual surface* the evidence was read off. We show why collapsing any pair of these makes a well-formed scholarly question unaskable, and we report the collision that arose when two of them were given the same name.
3. **A measured account of agentic contribution** (Section 5, Section 8). Model-extracted relationships are 890 of 510,905, or 0.17 %. We describe the mechanism that produced this ratio — a promotion gate encoded as a frozen set of trust classes rather than as a review convention — and characterise what agents did instead of asserting.
4. **An adversarial validation methodology whose gates attack withholdings** (Section 7.1). We report a gate, implemented independently of the pipeline it audits and equipped with a negative control, that falsified the stated reasons of 39 withheld rows using the project’s own released data, and we show why the correction changed no verse’s release status.
5. **A failure record offered as evidence about method** (Section 9). We catalogue the project’s withdrawals, refusals and self-deletions with the measurement that forced each, including two cases in which a refusal’s own decisive argument was subsequently found wrong and withdrawn in turn.

6. **The integrated graph and a research interface over it** (Section 11): 165,737 nodes and 510,905 relationships over 20,210 canonical mantras, exposed through an interface that renders the evidence typing at the point of reading.

1.4 What this paper does not claim

The corpus is four recensions, not “the Vedas”: no Padapāṭha, no Brāhmaṇa, Āraṇyaka or Upaniṣad, no Kṛṣṇa Yajurveda, no Paippalāda Atharvaveda, and no Sāmavedic *gāna* corpus are held (Section 3.1). No comprehensive human gold standard exists for the semantic layer; the reference sets that do exist are model-adjudicated or independent-source-adjudicated, and we use those terms strictly (Section 8.3). The construction was carried out by one person, so the human-decision arm of the workflow is a single individual’s judgement and is not independent of the project’s goals. Finally, the graph is not bootstrappable from a clean clone: we state precisely which parts are and are not (Section 8.7).

2 Background and Related Work

2.1 The Vedic corpus as a computational object

The four Vedic Saṃhitās are among the oldest continuously transmitted texts in any language, preserved orally with a fidelity that makes them unusually good material for textual computation and unusually unforgiving of sloppy addressing. Each survives in named *śākhās* whose verse inventories differ; a claim about “the Atharvaveda” that does not say Śaunaka or Paippalāda is not yet a claim about a corpus. The standard modern scholarly translation of the R̥gveda (Jamison and Brereton 2014) is explicit about this, and the nineteenth-century translations that remain the only complete English renderings reachable for parts of the corpus (Griffith 1896; Whitney 1905) each carry their own numbering conventions, which is a recurring source of alignment error (Section 7.2).

One reference work deserves particular mention because it is the direct predecessor of a layer reported here. Bloomfield’s *Vedic Concordance* (Bloomfield 1906) is an alphabetic index to every line of every stanza of the published Vedic literature, with an account of the variations across Vedic books — that is, a manual, exhaustive index of exactly the cross-textual repetition that Section 6 detects computationally. We did not use it as a gold standard, because the project does not hold a machine-readable form of it under rights permitting that use; we note the omission as the most obvious missing evaluation in this paper (Section 12).

For the Sāmaveda specifically, the relationship between the Ārcika verse text and its sung realisation is the subject of a substantial musicological literature (Howard 1977). That literature is the reason Section 7.1 is careful: the project records svara marks as printed and declines to decode them into pitch, and the decipherment authority it would need in order to do so responsibly is not held.

2.2 Sanskrit computational linguistics

The computational infrastructure for Sanskrit has matured substantially. The Digital Corpus of Sanskrit (Hellwig 2010–2021) supplies morphological and lexical annotation at scale; neural segmentation now handles the joint problem of compound splitting and sandhi resolution without hand-built features (Hellwig and Nehrdich 2018); the Treebank of Vedic Sanskrit (Hellwig et al. 2020) supplies human-validated dependency annotation over Vedic material specifically; and VedaWeb (Kiss et al. 2019) provides a research platform over morphologically and metrically annotated Old Indo-Aryan text. A digital Anukramaṇī (Akavarapu and Bhattacharya 2023) makes the traditional hymn-level index — seer, deity, metre — machine-readable for the R̥gveda.

VedGraph consumes rather than extends these. The treebank is the external annotation against which its reference set is constructed (Section 8.3); the Anukramaṇī is the source of its l1 attribution layer; VedaWeb supplies a parallel reading and a linguistic annotation layer. The contribution claimed here is not a new language resource but an architecture for holding claims of very different epistemic kinds in one queryable structure.

2.3 Knowledge-graph construction and refinement

Surveys of knowledge graphs (Hogan et al. 2021) and of graph refinement (Paulheim 2017) establish the vocabulary this paper uses and also its central gap: refinement is generally framed as improving *correctness* and *completeness*, with provenance treated as metadata about a dataset rather than as a filterable property of each assertion. Where per-claim provenance is modelled, the established mechanisms are reification and its successors — nanopublications (Groth et al. 2010), which package an assertion with its provenance and publication info as named graphs, and PROV-O (Lebo et al. 2013), which supplies a general vocabulary for entities, activities and agents.

VedGraph’s model is a considerably less expressive special case of what those standards can express, and deliberately so. Rather than a general provenance graph it stores four fixed, closed-vocabulary properties on every

edge (Section 4), which buys two things a general model does not: the axes are indexed, so “show me only source-explicit claims” is cheap rather than a join; and a closed vocabulary admits universal-quantifier invariants, so a missing or out-of-enum value is a build failure rather than an under-specified record. The trade is expressiveness for enforceability. We would expect a PROV-O export to be straightforward and have not built one.

2.4 Language models in extraction, and their failure modes

The programme of using language models to build and complete knowledge graphs is well surveyed (Pan et al. 2024). The failure mode this paper is organised around is equally well documented: generated text can be fluent, confident and unsupported (Ji et al. 2023), and — crucially for any design that hopes to let a model check itself — models do not reliably self-correct reasoning without external feedback, with performance sometimes degrading after a self-correction pass (Huang et al. 2024).

That last result concerns *intrinsic* self-correction — a model revising its own answer with no external feedback — which is not the regime described here: the attacking agent carries a different instruction and has read access to the store, which is the external-feedback condition that work excludes. It is therefore a motivation for the division of labour reported here rather than evidence that the division works, and we are aware of no result that predicts it will. VedGraph does not ask a model to assess its own output; it asks a *differently-instructed* agent to attack that output with a stated hypothesis, and it requires the attack to be expressed as a measurement against the store rather than as a judgement (Section 5.4). We are explicit in Section 9.3 that this does not achieve statistical independence, and that a correlated failure across a shared model family would be invisible to the whole regime.

On agent architectures, interleaving reasoning with tool-use actions (Yao et al. 2023) is the pattern the construction runtime instantiates, but *only* as a pattern: no orchestration framework was used, and we verified that by inspecting the dependency manifests (Appendix C). We mention the literature to place the work, not to claim a framework the repository does not contain.

2.5 Text reuse detection

Detecting reuse in historical corpora is an established line with mature tools: scalable passage-cluster detection over large noisy collections (Smith et al. 2013), intertext detection tuned for classical poetry (Coffee et al. 2013), and a general algorithm suite for historical reuse (Büchler et al. 2014). Our candidate-generation strategy — banded MinHash over character n -grams, then a composite score — is conventional against that background, and Section 6.3 reports the parameters rather than claiming novelty for the method.

Two things here are less conventional. First, the comparison *surface* is treated as a research object rather than as preprocessing: six named match levels are kept distinct and the code records which one a given edge was matched on, because a script-folded match and a byte-identical match assert different things (Section 6.1). Second, *direction*. The reuse literature generally establishes direction from external chronology. Because Vedic relative chronology is contested, this project measures a structural signal instead — compilation asymmetry (Section 6.8) — and validates it per scope, accepting it where it recovers a tradition’s own claim at 0.991 consistency and refusing it where it cannot separate the sides.

The corpus’s formulaic character connects to oral-formulaic theory (Lord 1960), which supplies the reason a formula layer is worth building at all: in an orally composed and orally transmitted corpus, a repeated multi-word span is a unit of composition rather than an accident. We use that as motivation and make no claim to contribute to the theory.

2.6 Evaluation without a gold standard

Reporting agreement figures properly requires care about what the coefficient measures and what population it was computed over (Artstein and Poesio 2008). The situation in this project is more constrained than the usual one: there is no human annotation of any VedGraph claim at all, so no inter-annotator coefficient over its assertions is computable even in principle (Section 8.3). What exists is an external, human-validated annotation of the *text* (Hellwig et al. 2020), against which claims can be adjudicated mechanically. The project names that arrangement with its own term, and refuses to call it gold. Everything we report about the semantic layer’s accuracy is bounded by that.

Finally, the data-management principles the project’s registries implement — rights and provenance recorded per artefact, identifiers stable and resolvable, machine-readable scope — are those of the FAIR framework (Wilkinson et al. 2016), with the important qualification that reusability here is constrained by third-party rights the project does not hold (Section 13).

Table 1. The canonical corpus. Each work is one recension of one Veda; the scope limits in the last column are stored as queryable properties on the `Work` node, not only as prose. “Rendered” counts mantras reachable from at least one `Translation` node, which for the *Sāmaveda* means reused *Ṛgvedic* renderings rather than *Sāmavedic* translations (Section 7.2).

Work	Recension	Mantras	Rendered	%	Explicitly not held
<i>Ṛgveda</i>	Śākala	10,552	10,516	99.7	no second recension; no <i>Padapāṭha</i> ; no <i>Brāhmaṇa</i> , <i>Āraṇyaka</i> or <i>Upaniṣad</i>
<i>Atharvaveda</i>	Śaunaka	5,839	5,788	99.1	Śaunaka only — <i>Paippalāda</i> absent; no <i>Gopatha Brāhmaṇa</i>
<i>Yajurveda</i> (Śukla)	Vājasaneyi Mādhyandina	1,975	1,950	98.7	Śukla only — the entire <i>Kṛṣṇa Yajurveda</i> absent; no <i>Kāṇva</i> ; no <i>Śatapatha</i>
<i>Sāmaveda</i>	Kauthuma Ārcika	1,844	173	9.4	<i>ārcika</i> only — all four <i>gāna</i> collections excluded; no second recension
Total		20,210	18,427	91.2	

3 Corpus, Sources and Identity

3.1 Recension is part of the corpus definition

“The Vedas” does not name a corpus precisely enough to build on. Each Veda survives in multiple *śākhās* whose verse inventories and orderings differ, and each *Samhitā* sits beneath a stratified literature — *Brāhmaṇa*, *Āraṇyaka*, *Upaniṣad* — that a reader may or may not expect to be included. An absence measured in one recension is not an absence from the tradition.

VedGraph therefore defines its corpus as four `Work` records, each one recension of one Veda, and stores the exclusions as machine-readable properties rather than as prose (Table 1). Each `Work` carries `scope`, `excluded_corpora` and `completeness` fields; a live invariant fails the build if either of the first two is null. The reason the field exists is recorded in the model itself: until the scope statement reached the graph, a consumer querying `work_name` read “*Samaveda Samhita*” and was misled by what the project calls the most misleading string in its own product.

The *Sāmaveda* entry is the sharpest case. The *Ārcika* verse collection held here is roughly 1,875 verses; the *gāna* collections — *grāmageya*, *āraṇyakageya*, *ūhagāna*, *ūhyagāna* — are a parallel and larger body of about 2,639 chants. The registry states the consequence flatly: the sung dimension is not a decoration on the verse skeleton but the reason the *Sāmaveda* is a distinct Veda rather than a *Ṛgvedic* excerpt, so any claim to “have the *Sāmaveda*” while holding only the *Ārcika* is false. Fourteen such refusals are recorded across the four works.

3.2 Rights attach to files, not to hosts

Three registries describe provenance at three different granularities and are allowed to disagree: `sources.yaml` records the host, `source_artifacts.yaml` the individual file and its licence, and `text_versions.yaml` the text layer. Thirteen numbered policy rules govern the reconciliation; the load-bearing ones are that on conflict the more restrictive statement controls, that absence of a licence field is never a grant, that chain of title is followed to its root rather than to the nearest licence, and that a permissive status is never inferred but only evidenced. The distribution of statuses is itself informative: at file level, 15 of 53 artefacts are `UNKNOWN`.

One consequence is worth stating because it constrains what this project could ever publish. The *Ṛgvedic* primary text is CC BY-NC-SA, the *Sāmavedic* and *Yajurvedic* texts are CC BY-SA, and the *Atharvavedic* text is reference-only; the registry concludes that a combined four-Veda release is not licensable as a single adapted work from these sources. The graph is buildable and queryable; the corpus is not redistributable as one artefact. Section 13 states what is and is not available.

A separate provenance audit over the raw snapshot store verified 1,347 metadata records against their payloads with zero hash mismatches, zero missing payloads and zero orphans. That audit names its own limits in terms this paper adopts: hash verification proves a snapshot is unaltered since retrieval and says nothing about whether retrieval was permitted, nor whether the digest was pinned against the right artefact. Integrity and scope are independent properties.

3.3 Identity

The scheme. Let $\mathcal{N}$ be a fixed namespace UUID, constant for the life of the protocol. For a passage p , let

$$\kappa(p) = \langle \text{veda}, \text{recension}, \ell_1(p), \ell_2(p), \dots, \ell_k(p) \rangle$$

be the tuple of its work’s declared structural coordinates, where k is a property of the work rather than of the schema: the *Ṛgveda* uses $\langle \text{maṇḍala}, \text{sūkta}, \text{mantra} \rangle$, the *Vājasaneyi Samhitā* $\langle \text{adhyāya}, \text{mantra} \rangle$, and the *Sāmaveda*

up to four levels beneath a *named* collection. A total, injective serialisation u maps $\kappa(p)$ to a canonical URN, and identity is

$$\text{id}(p) = \text{uuid5}(\mathcal{N}, u(\kappa(p))), \quad u(\kappa(p)) \in \text{urn:vedagraph:}\Sigma^*.$$

Nothing else may enter. The specification enumerates the exclusions: not the source artefact, its URL, its revision or its licence; not the `TextVersion`, the script, the accent notation or the text itself; not the build, the parser version or the segmentation policy; not sequence position anywhere. ADR-002 states the rule as “citations and source locators never determine identity”, and a companion decision makes the corollary testable: because identity derives from the citation hierarchy alone, changing the base edition changes which text is displayed and nothing else.

We verified the scheme by recomputing $\text{id}(p)$ for every passage in all four canonical builds: 22,537 passages pooled, with zero duplicate keys, zero duplicate URNs, zero duplicate UUIDs and zero mismatches against the shipped corpus. Human-readable keys (VG:RV:SAK:M01:S001:V001) are padded for lexical sorting and are a join and display surface; URNs are unpadded and are the hash input. Only the URN may never be re-spelled.

Why identity excludes content. Two hash families exist and are deliberately disjoint. Content digests — `content_sha256` on a stored reading, `checksum_sha256` on a source artefact — serve integrity, never identity. Putting a content hash into the identifier would make every re-encoding a new entity and would couple canonical identity to a mutable upstream wiki.

The interesting consequence is that the obvious identity check is vacuous. Recomputing a UUID from its URN is a closed loop: it proves the URN hashes consistently and says nothing about what the URN *addresses*. Because a `Passage` stores no text, when a segmentation change moved five Sāmavedic referents under unchanged UUIDs, every gate in the build reported success. The project’s response was a second, separate mechanism: a *referent-drift guard* that fingerprints each key’s text twice, once over the bytes the source spells and once over a normalisation-independent comparison surface, and compares both against the previous release’s fingerprint for the same key. A difference is adjudicated into one of five verdicts — unchanged, intentional correction, drift, removed invalid passage, newly discovered passage — and drift blocks release.

Even that guard needed hardening. Text equality alone is insufficient, because of 1,844 released Sāmavedic keys only 1,651 have a distinct comparison digest: 192 equivalence classes covering 385 keys share one, and 174 of those classes share a byte-identical raw digest too. The guard therefore requires three fields to agree — the comparison digest, the source locator, and the source verse marker — before it will call two observations the same occurrence.

3.4 Normalisation is a comparison instrument

The governing rule is stated as a campaign-wide invariant and appears verbatim in the owner decisions, in the code that implements the folds, in the public API’s evidence vocabulary, and in the project’s README:

Unicode normalization, accent folding, punctuation folding, whitespace folding, script transliteration, sandhi-normalized comparison, case folding and OCR normalization may all **generate candidates**. None of them alone may establish that two things are the same canonical passage, the same entity, the same recension, a `VARIANT_OF`, or a text reuse, and none may license a canonical merge.

Three states are kept apart by name: *normalized equivalence* (the strings fold together), *canonical identity* (they are the same thing, on independent evidence), and *textual variant* (a real difference in transmission). Same folded text at different coordinates is explicitly not the same passage.

Comparison surfaces are therefore named and ordered rather than implicit. Six *comparison forms* run from the identity function on stored text through NFC, accent-preserving normalisation, accent stripping, a search surface, and a cross-script surface. Stored text is never replaced: `text_original` and `text_nfc` are separate fields so the original is recoverable, and a comparison surface may be lossy where stored text may not.

The folds encode real philological judgements, and the project records the ones it refuses to make. Avagraha is a genuine orthographic sign and is deliberately not folded away, at a measured cost of 46 of 116 residual unclassifiable Yajurvedic pairs. A private-use code point occurring 20 times in one witness is deliberately not folded, because reading it as an anusvāra from context is a transcription judgement rather than a normalisation, and it is registered as a source defect instead. Conversely one fold is knowingly lossy: the entire lateral series collapses onto a single sentinel because U+1E37 denotes the vocalic *l* in one source convention and the retroflex *ḷ* in another — the same code point, opposite phonemic class — so no global table can be correct for both. Splitting it was implemented, measured, and reverted when it regressed a verse from a classified comparison to an unclassified one.

A fold inside a digest is not free. Because the referent guard’s comparison digest is computed through the search surface, adding a rule to the transcription table changes the digest of already-released keys, and the guard then reports drift where none occurred. This was measured — one rule moves five Yajurvedic keys, others four

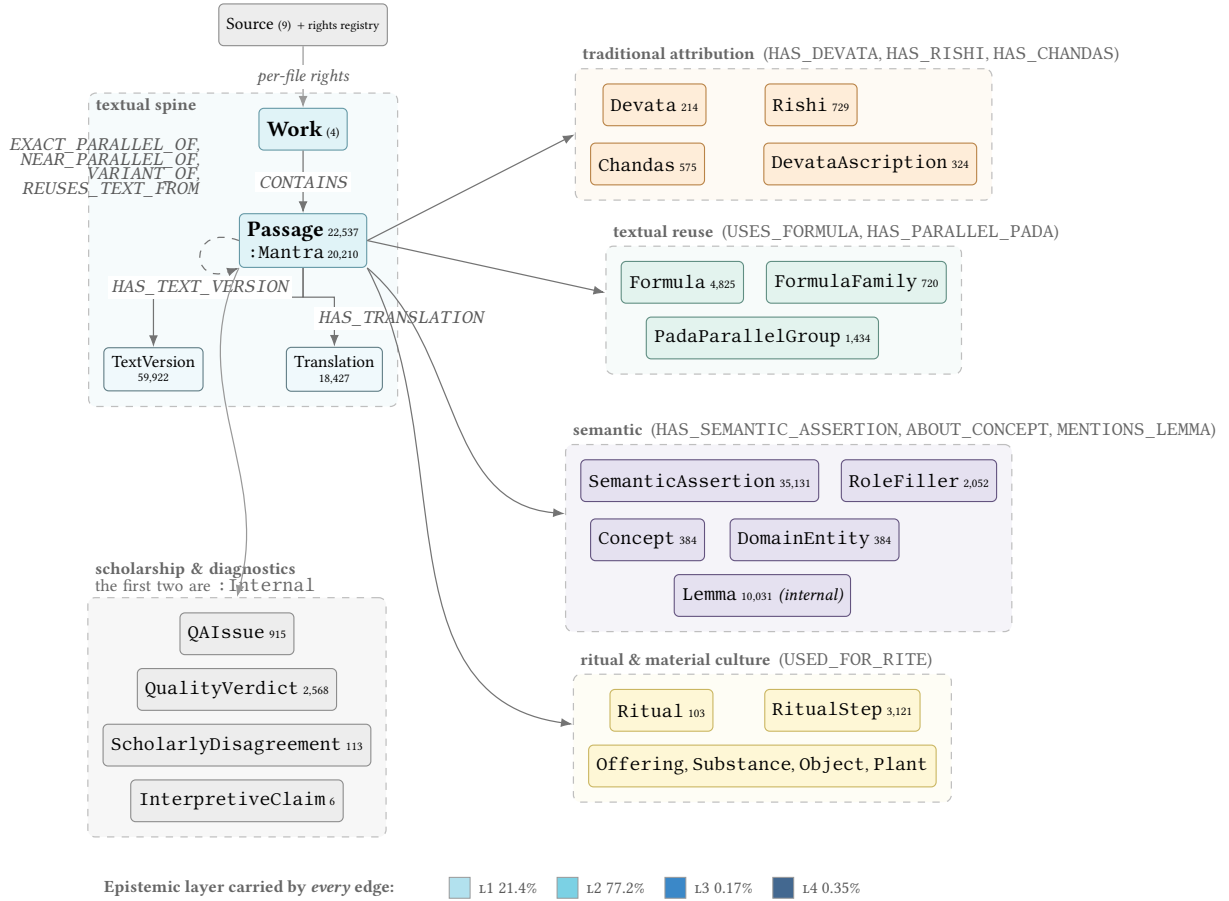


Figure 1. Core graph ontology, grouped by object family. An overview: 55 node labels and 81 relationship types are in use, and the full catalogue is [Appendix A](#). The figure shows one labelled edge per family rather than every predicate, because the alternative is unreadable. Counts are from the fact freeze at 6bf220c.

and one more — and resolved by splitting the fold tables in two: a frozen table inside the digest, and a free table for comparison only. The code comments the split as the owner’s invariant seen from the other side: a comparison fold must be free to change while identity is not.

3.5 Recorded identity defects

The project’s identity work is best characterised by the defects it caught rather than by the scheme in the abstract. Three are worth reporting.

A disputed name kept out of identity. The Sāmavedic Ārcika is addressed by named collections, never by a bare ordinal, because the first slot’s value 2 denotes the Āraṇyārcika in one e-text lineage and the Uttarārcika in six independent witnesses. The blast radius was measured before the decision: 1,280 of 1,875 verses are mis-addressed under the rejected reading. The four collection extents agree across every witness while the extent of the disputed name “Pūrvārcika” does not, so the extents entered identity and the name did not.

A recorded reason that was backwards. The stated cause of a Sāmavedic withholding was corrected when re-measurement showed that for three of four groups the selected witness prints the *daśati* heading and the second witness omits it — the opposite of what had been recorded. The data were right; the explanation was not.

An edition’s hymn order is data, not a parser conditional. Griffith prints the Vāḷakhilya as R̥gveda Book 8 hymns 93–103 where the base edition places them inline at 8.49–8.59. Building without that mapping did not produce missing translations; it produced wrong ones. The decision record states the principle the whole identity layer turns on: a gap is honest, a silent misalignment is not.

Table 2. Relationships by epistemic layer, measured on the live graph at 6bf220c. The layer is a stored, indexed edge property, so this distribution is a query rather than an estimate. Two pipeline generations wrote the field with and without the L-prefix; the legacy spellings are folded here and reported unfolded in the fact freeze.

Layer	Meaning	Edges	%
L1 source-explicit	a source states it	109,298	21.39
L2 deterministic-derived	a reproducible rule over stored text or over L1	394,540	77.22
L3 LLM-extracted	a language model read evidence and proposed it	890	0.17
L4 interpretive-claim	a reading held by this project or a named commentator	1,787	0.35
untyped (a recorded defect)	no layer recorded; see Section 9.3	4,390	0.86
Total		510,905	100.00

4 The Evidence-Typed Data Model

4.1 Four axes, deliberately not collapsed

The object families are shown in [Fig. 1](#). Every relationship in VedGraph carries, as stored properties, the answers to four different questions. They are orthogonal, and the model’s central claim is that collapsing any pair makes a well-formed scholarly question unaskable.

(1) How did this claim come to be? The *knowledge layer* is a four-member closed vocabulary, described in the code as “never collapsed, never inferred from a score”:

(2) How should a reader grade it? The *quality tier* is a reader-facing letter derived from the layer by a total bijection ($L1 \rightarrow A$, $L2 \rightarrow B$, $L3 \rightarrow C$, $L4 \rightarrow D$). The module states the rule as: tier is how a claim was reached, never how confident it sounds. The tier exists separately from the layer because one further transition acts on it — a model-extracted claim that no reviewer has accepted stays in L3 but drops to TIER D, since TIER C is defined as model-extracted evidence that survived review and a candidate has not had any.

(3) What is the claim’s reach over the passage? *Attribution precision* distinguishes a per-passage statement, a label *inherited* from an enclosing container, the passage’s own wording, and edges that are not attributions at all. This is the axis that matters most for counting. The Anukramaṇī names a deity for a *sūkta*; projecting that label onto each of its mantras is what makes “passages of Indra” work at all, but a reader who cannot see the difference reads tens of thousands of inherited labels as per-verse statements. The model records the consequence: “How many mantras invoke Indra?” has at least four honest answers — strict source-stated, container-inherited, textual mention, and semantic invocation — and collapsing any pair of them makes one of the four unaskable. The live distribution shows how unevenly the corpus behaves: every one of the Vājasaneyi Saṃhitā’s 2,240 seer attributions is source-stated per verse, while every one of the Atharvaveda’s 5,084 is a *sūkta* label projected downward, so a per-verse Atharvavedic seer claim cannot be made from this corpus at all.

(4) Off which textual surface was the evidence read? A rule fired over Griffith’s 1896 English is exactly as reproducible as a rule fired over the Sanskrit, so the tier cannot separate them — and the difference matters enormously, because one measures a Victorian translator’s word choice and the other measures the text. The project discovered this the expensive way: 21,246 of 47,542 ABOUT_CONCEPT edges had fired on an English keyword with no Sanskrit evidence and carried an identical grade to edges matched on the verse itself. Those edges were later withdrawn, and a separate *evidence surface* axis was introduced so the distinction is filterable rather than archaeological.

4.2 A name collision, and what it cost

The fourth axis is instructive as a failure. Two different enumerations in the codebase are both called `EvidenceBasis`. One describes the textual surface the evidence was read off; the other describes the kind of act that produced the claim. Their value spaces are disjoint. Reading the stored graph property straight into the second enumeration — the natural thing to do, given the shared name — mapped *every* attribution edge in the corpus to UNKNOWN: all 17,889 seer edges, all 16,298 metre edges, all 10,558 deity edges. The failure was silent, because UNKNOWN is a legitimate value.

The remedy is worth reporting because it generalises. Rather than renaming one enumeration, the project added a test that enumerates the property’s entire observed value space in the live store and asserts it is a subset of

the surface vocabulary. The test’s header records why the check is shaped that way: this project has twice certified an absence by asking the wrong question, once against the wrong surface and once with the wrong vocabulary. Grepping for the value you expect confirms your expectation; enumerating the value space does not.

4.3 The lifecycle: a model may write exactly one status

The constraint that keeps `l3` small is not a review convention. It is encoded as data. Assertion state is a three-member vocabulary — candidate, accepted, rejected — and the set of trust classes permitted to be written as *accepted* without review is a frozen set containing exactly deterministic-derived and source-explicit. The comment beside it is explicit that this is the policy expressed as data rather than as a convention. The constructor raises if a model-derived record is marked accepted, and raises again if a record claims model provenance without naming a model.

The governing decision record, ADR-014, states the rule and three consequences that are each enforced in code:

A model may write exactly one status: CANDIDATE. Every other status is written by a deterministic stage or by a person. ... Evidence is checkable or the claim is discarded: a cited token the model was never shown is a fabricated citation, it is decidable, and it is rejected. ... The predicate vocabulary is closed, and the refusals are named. ... Confidence is a floor inside a predicate’s rule and nothing else: acceptance requires that the predicate has been *unlocked* by a measured gold precision of $\geq 95\%$; until then, a confidence of 1.0 is accepted at the same rate as 0.4 — not at all.

Its stated rationale is the reason we treat confidence-thresholding as unsound in this setting: the confidence score is produced by the same process that produced the claim, so it carries no information independent of the claim, and thresholding on it selects for fluency rather than for truth. The unlocked-predicate set is empty by construction and remains empty; consequently no semantic assertion has ever been auto-accepted.

Two refinements are worth noting. First, five predicates that a model might naturally want — `SYMBOLIZES`, `REPRESENTS`, `IS_GOD_OF`, `MEANS`, `CAUSES` — are members of the enumeration *so that a proposal naming one is rejected as a policy decision with a recorded reason*, rather than failing schema validation as an unknown string. The difference matters when reading a rejection report. Second, the strongest review state the graph has ever recorded is `MODEL_ADJUDICATED`, glossed in the API as “a model re-read the passage and accepted this edge with a stated reason. This is the strongest review state in this graph and it is not human review.” The count of human-reviewed edges is zero, and the product says so.

4.4 Typed absence

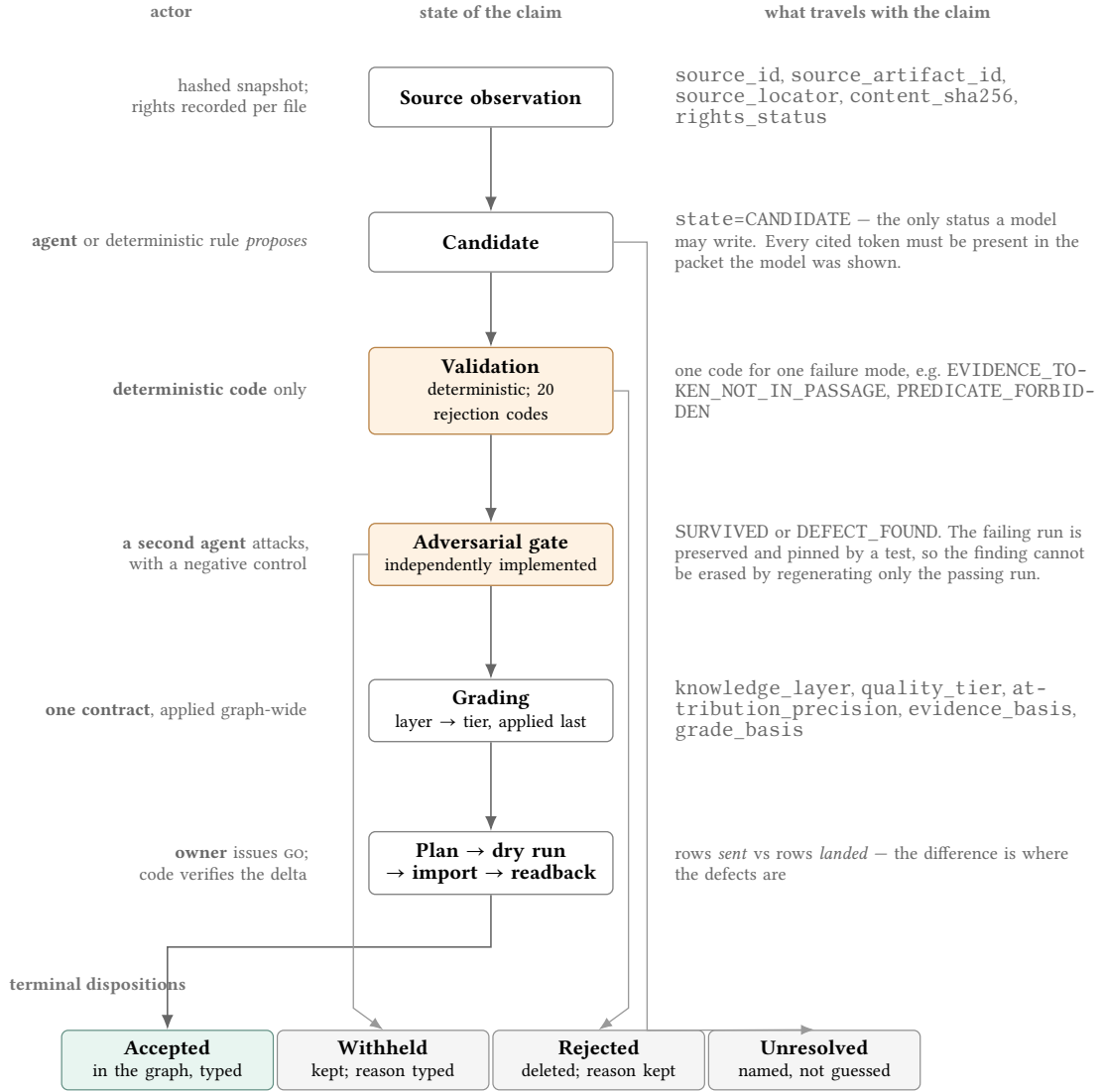
Figure 2 sets out the whole lifecycle, from a source observation to one of four terminal dispositions. The model treats an untyped absence as a defect. Four distinct mechanisms record *why* something is not present, each scoped to a layer: a reason string per unmapped predicate, emitted into the load report alongside a separate count of predicates that are neither mapped nor explicitly withheld (“not mapped” and “nobody looked” are indistinguishable in a coverage number, and only one of them is acceptable); a typed node property with a declared, closed class space, used for Sāmavedic notation (Section 7.1); typed non-resolution rows in the lexical layer, where 711 unresolved tokens each carry a linguistic reason rather than a queue state; and edge-type-keyed withholding, where near-parallel pairs are held back from a reuse claim with the reason recorded on the type.

The invariant that makes the Sāmavedic case work is published as a measurement rather than asserted: the count of verses with no notation disposition at all must be zero, because an untyped verse is one a reader cannot tell apart from a verified zero.

4.5 The public boundary, and a label that does two jobs

Product and internal knowledge share one database. Diagnostic nodes — the repository’s assessments of its own passages — carry a marker label so that a single filter, written once, excludes them. The design note is that sprinkling an exclusion across thirty queries would be the same policy enforced thirty times and wrong the first time a new diagnostic label appeared.

The marker turns out to mark two different kinds of node, and the distinction is not drawn by the label itself. It marks the diagnostic layer, which no researcher should traverse into, and it marks *sub-entities* — text versions, translations, lemmas, sources — which must stay reachable, since a passage that could not reach its own text would be useless. The broad form of the obvious leakage check therefore reports 70,559 “leaks”, every one of them legitimate, and an auditor running it would file a false critical finding. The invariant is consequently written against the diagnostic labels by name rather than against the marker.



A withheld claim is neither a rejected one nor a statement that the thing does not exist. Each withheld population has a closed class space, and the number of rows carrying no disposition at all is itself gated at zero — an untyped absence is one a reader cannot distinguish from a verified zero.

Figure 2. The epistemic pipeline: how a claim acquires its typing and what it can terminally become. The centre column is the state of the claim, the left gutter names the actor, and the right gutter names what travels with the claim at that point. Only one of the four terminal dispositions puts a claim in the graph; the other three are recorded rather than discarded, because an absence a reader cannot distinguish from a verified zero is the most misleading thing this graph could publish.

A related caveat is structural and is worth stating for anyone building on such a graph: the separation is query-side, not structural. An unfiltered one-hop traversal from any passage returns tens of thousands of text-version and translation nodes. The mitigation the project adopts is to serve a catalogue of vetted queries rather than the database.

4.6 Invariants

Twenty live invariants and twelve integrity gates run against the loaded store. The design principle is stated in the invariant suite’s header: each check is phrased as the defect it detects rather than as the desired state, because each corresponds to a defect this repository actually shipped. Representative members include: no edge without a quality tier; no edge without a grade rationale; no unlabelled node; no product node reaching a diagnostic node; no work without a scope statement; no attribution-precision value outside the declared enumeration; no predicate carrying more than one run identifier, since that would mean part of the graph is reproducible from no committed artefact.

Two features of this suite bear on method. First, one gate had been rewritten twice and both earlier versions were untestable — they computed the set of declared relationship types as the union of the ontology *and the graph*, then asked which graph types were missing from it. Nothing can be. The gate reported zero undeclared types for an entire wave while eleven predicates went unclassified. The code now carries the lesson as a comment: the data may not declare itself valid. Second, a universal-quantifier invariant over attribution precision could only be written safely once the vocabulary had a member meaning “this edge is not an attribution” — an invariant asserting that every edge carries a property must first be able to say what the property means on every edge.

5 Agentic Construction

5.1 What “agentic” means here, and what it does not

The construction ran in a coding-agent runtime: a language model with filesystem, shell and database access, directed by one human owner, writing and executing code as well as reading sources. It did *not* use ReAct, AutoGen, LangGraph, CrewAI, Reflexion or any comparable orchestration library; we searched the project’s dependency manifests and found none, and we mention the families only because the related-work section discusses them (Section 2). Orchestration here is the project’s own artefact protocol — briefs, staging directories, proposal files, gates, rulings and receipts — rather than a framework.

Two consequences follow. The workflow is inspectable, because each step left a file. And it is not a reusable system: what generalises is the protocol, not an implementation.

5.2 The construction pipeline

Figure 3 shows the pipeline the artefacts describe. Its spine is the same for every layer: a source is snapshotted and hashed; a canonical corpus is built deterministically and given identity; a layer is proposed into a *staging* directory that is validated and gated before any graph write; the owner issues a GO against a hashed plan; the import executes; and a readback compares what was sent with what landed.

The staging step is the load-bearing one. A layer’s proposal is a directory of row-level JSONL plus a manifest, validated by a contract that reports *its own coverage* — the validator states how many rows each check evaluated, so a check that silently skipped its population is visible as a coverage shortfall rather than as a pass. During the completeness campaign the rule was stated flatly: nothing is imported until a readback.

5.3 The import protocol

Every graph mutation in the later campaigns followed a five-step protocol, and its steps exist because each was once skipped:

1. **Plan.** A hashed artefact enumerating the exact nodes, edges and property writes, with the expected census delta.
2. **Dry run.** The plan re-computed against the live graph, emitting a GO or the reason it cannot. Dry runs were regenerated whenever the tree changed, because a dry run computed against a different state is not a dry run.
3. **Owner GO.** A recorded human decision against a specific plan hash.
4. **Execute.**
5. **Readback.** An independent re-measurement comparing rows *sent* against rows *landed*, with the census delta checked against the promise.

The protocol’s value is measurable in what it caught: a first import that over-wrote by 5,930 edges, a second that left 12,033 nodes unreachable, a third that landed 13,187 elements the domain rules excluded, and one that silently overwrote 102 curated display labels. Roughly nine plan/dry-run/GO cycles preceded one clean readback in the largest wave.

It also failed in an instructive way. One readback printed `READBACK_CLEAN` having compared zero rows against zero rows, and a separate import’s own readback reported success while two graph-wide contracts were unmet — the scorecard, not the importer, found 1,136 edges that were ungraded, unlabelled and unmarked. The lesson the repository records is the one we would generalise: *an importer’s own success report is not evidence*.

5.4 The multi-agent structure, as the artefacts show it

The clearest instance is the final closure sprint, and we describe it using the repository’s own names rather than inventing roles.

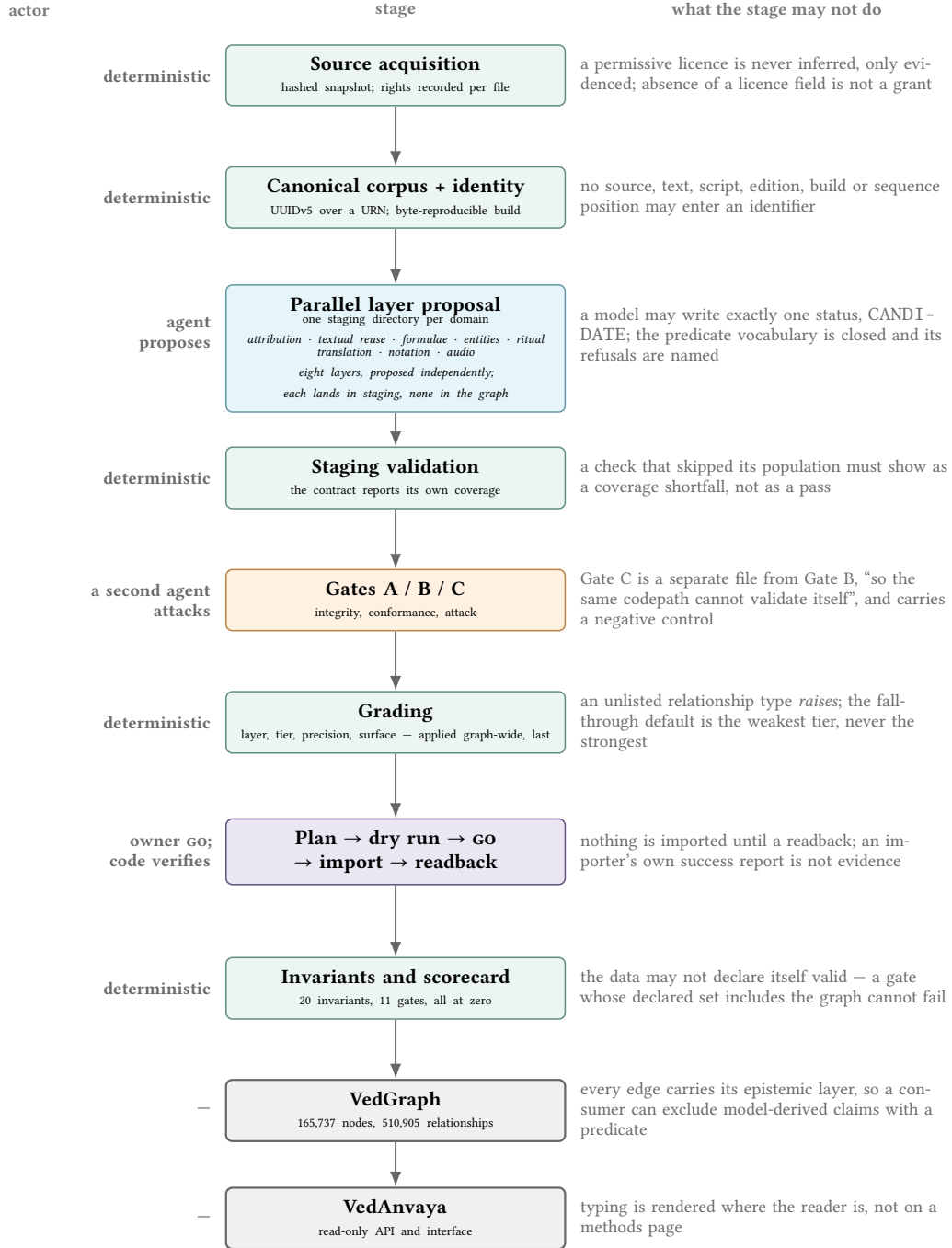


Figure 3. The construction pipeline. The left gutter names the actor for each stage; the right gutter names what that stage is forbidden to do, which is where the method actually lives. Agents propose into staging; deterministic code validates, grades and verifies; the owner issues the one decision that lets a mutation run.

Forty-three remaining gaps were partitioned across **seven domain agents**, one owner each, under a stated rule: every implementation-fixable row has exactly one owner agent, and no row is unassigned. The agents are named by domain — AGENT_1_FORMULA_CROSS_VEDA (3 gaps), AGENT_2_ENTITY_DEVATA_IDENTITY (12), AGENT_3_MORPHOLOGY_SEMANTICS (7), AGENT_4_RITUAL (4), AGENT_5_QUALITY_REFERENCE (6), AGENT_6_PRODUCT_SURFACE (5) and AGENT_7_TRANSLATION_SAMAVEDA (6). Each produced measurements, row-level evidence and a file named `proposed_mutations.json`. *None of them wrote to the graph.*

An eighth agent had no gaps assigned. Its artefacts are an audit of the other seven, its recorded database access is `READ_ONLY`, and it is the adversary in the pattern. A **lead** then reconciled proposals and findings into an integration plan, a set of rulings and a receipt.

Figure 4 shows the resulting cycle. Its most important property is visible in one ruling. Agent 3 proposed re-anchoring a relationship so that role fillers hang from semantic assertions. Agent 8 measured that the change would create 2,052 endpoint-signature violations, because the declared signature expects a passage endpoint and zero of the assertion nodes carry that label. The lead verified the measurement independently, and then ruled neither “approved” nor “rejected” but *approved with the ontology moving in the same change* — because the migration card had declared the new signature all along, and either half alone would leave 2,052 violations. A test now pins the signature to the card.

Three features of that ruling recur across the corpus of rulings and are, we think, the transferable part:

- **The adversary measures rather than opines.** Agent 8’s findings are counts against the live store, so they can be re-run. One of them — that a registry-status check “only compared a graph measurement to a pre-declared integer; it never read the status field” — is a defect in a *check*, not in data.
- **The integrator re-measures before ruling.** The lead’s rulings record `lead_measurement` separately from the agent’s claim, and in one case correct the agent’s figure: an agent measured the live layer at 4,865 unreviewed assertions while its own staged delta would take the post-integration value to 35,139 — a prediction, against the 35,131 the layer holds today.
- **Trades are recorded, not netted.** One ruling notes that closing a coverage gap multiplies another gap’s unreviewed population sevenfold and deepens a third, and records the trade instead of reporting the improvement alone.

The adversary also found defects nobody had a gap for. It measured that the exported public artefact under-declared its own inputs — 31 of 36 exported labels undeclared, 129,691 edges and 32,044 node rows invisible to the hash that was supposed to cover them, and 4,016 public nodes dropped silently. And it found a user-visible search defect by querying rather than by reading code: 5,123 of 5,839 search derivatives in one corpus carried a private-use sentinel character, so a reader searching for *rtasya* got zero hits while the sentinel form got 94.

5.5 Five hostile rounds

Before release, five further adversarial rounds ran over the candidate (Section 9). The pattern to report is that later rounds found defects in earlier rounds’ work, including one round that closed two critical findings its own hostile pass had raised against itself. The first round’s finding was structural: 30,266 semantic-assertion nodes carried no assertion identifier and fell through to their source passage’s canonical key, collapsing two public objects onto one and silently dropping assertion-to-passage edges as self-edges.

We are careful about what this shows. It shows that a differently-instructed pass finds what an earlier pass missed. It does not show statistical independence: the passes share a model family, and a correlated blind spot would be invisible to all of them (Section 9.3).

5.6 Three gates per domain

Fourteen domains passed through a three-gate structure, adopted after four of four domains failed a third gate that had not previously existed. Gate A verifies the artefact’s own integrity — that its manifest checksums hold. Gate B verifies the rows against the live graph: coverage, addressing, provenance, and the central importability policy. Gate C attacks. In the case we treat at length (Section 7.1) Gate C was implemented in a separate file from Gate B explicitly “so the same codepath cannot validate itself”, named the fields it refused to read, re-implemented the normalisation from prose rather than reusing the pipeline’s code, and carried a negative control.

The owner-decision chain that produced Gate C is itself part of the method. A condition required that “existing validation gates pass”; the eligibility record showed one gate unknown and the other not run. The condition was recorded as *unmet*, with the reasoning that reading “gates” as the one gate that already existed would clear the board by narrowing a word, and that the conservative reading is the one that cannot manufacture a closure. The owner confirmed the conservative reading, the gates were built, and the new gate landed a real defect on its first run.

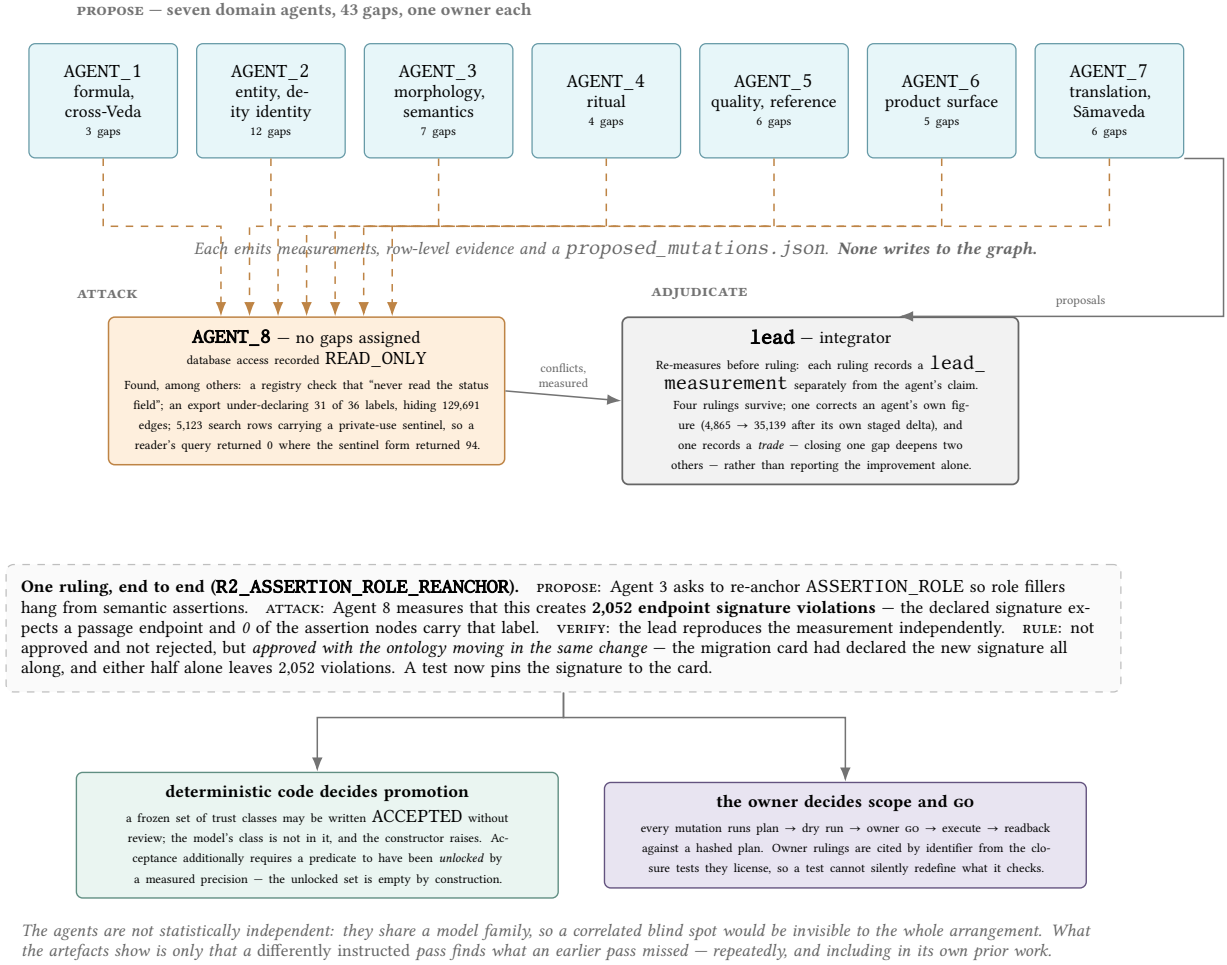


Figure 4. Propose → attack → adjudicate, using the repository’s own agent names and counts. Seven domain agents own 43 gaps between them and write no graph mutations; an eighth agent has no gaps and read-only database access, and audits the other seven; a lead re-measures before ruling. The boxed example is one ruling end to end. The caveat at the foot is not decorative: these agents share a model family and are not statistically independent.

5.7 Model output is confined by mechanism

Section 4.3 states the policy; here we note where it bites. The extractor’s validator rejects with one code per failure mode, and its central check is whether a cited token was present in the packet the model was shown — a fabricated citation is decidable and is discarded. The predicate vocabulary is closed, and five predicates a model would naturally reach for are members *so that proposing one is refused by name with a recorded reason*. Acceptance requires a predicate to have been unlocked by a measured precision against gold; the unlocked set is empty by construction and remains empty, so no assertion has ever been auto-accepted.

Runtime provenance is recorded rather than assumed, on a rule adopted after a concrete failure: earlier pilot output had been recorded as model-generated when it was in fact produced by regular expressions. The decision record calls that “provenance that looked like evidence and was not”, and notes that a wrong runtime string is the same defect in one field. Self-reported model identity is classified as unattested, and provider-attested build metadata is marked unavailable because no provider supplies it.

5.8 Who did what

The distribution in Table 3 is the answer to RQ2. Agents are present in almost every row and decisive in none of the rows that define truth: identity, promotion, graph mutation and release certification are deterministic code or a human decision, every time.

Table 3. Division of responsibility by construction stage. • = performs the work; ◦ = participates but cannot decide; — = not involved. The rows that determine what the graph *asserts* — identity, promotion, mutation and certification — have no filled agent cell.

Stage	Det. code	Agent	Owner	Where the decision is recorded
Source discovery and rights	◦	•	•	rights registry, per file; owner rules on conflicting statements
Snapshot and hashing	•	—	—	content-addressed store, 11 required sidecar fields
Canonical parse and identity	•	—	—	UUIDv5 over a URN; no model in the path
Ontology and vocabulary design	◦	•	•	decision records; closed enums with named refusals
Candidate generation	•	•	—	staging rows; deterministic rules and model proposals both land here
Validation of candidates	•	—	—	20 rejection codes, one code per failure mode
Adversarial audit	◦	•	—	a read-only agent with no gaps assigned; findings are counts, re-runnable
Reconciling conflicting proposals	◦	•	•	lead rulings, each recording its own independent measurement
Promotion candidate → accepted	•	—	◦	a frozen set of trust classes; the model’s class is not in it
Graph mutation	•	—	•	plan → dry run → owner GO → execute → readback
Readback and invariants	•	—	—	rows sent vs rows landed; 20 invariants, 11 scorecard gates
Scope and withholding rulings	—	◦	•	owner decisions, cited by identifier from the closure tests they license
Release certification	•	◦	•	certification document; withheld once on a single blocker
Manuscript drafting	—	•	•	this paper; claims and interpretation are the author’s

6 Connection Discovery

The Vedic corpus repeats itself heavily: whole verses recur across Saṃhitās, and shorter spans recur within them. Recording that as a single “similar to” relation would destroy the only distinctions that make the layer usable. This section states the algorithms, their parameters and their refusals.

6.1 Comparison surfaces are a ladder, and it is not nested

Before any similarity is computed, a pair of texts is placed on a ladder of six *match levels*, ordered by how much a match at that level asserts: source-exact, Unicode-normalised, punctuation-normalised, accent-insensitive, script-folded, and sandhi-insensitive. A source-exact match says the two editions print identical bytes; a sandhi-insensitive match says only that the two verses have the same letters in the same order once script, accent, punctuation and word division are all set aside. Both are real findings, they are not the same finding, and the code states that collapsing them would be the single most misleading thing the layer could do.

Two consequences are enforced. Which levels are *reachable* depends on the pair: a Latin-script text and a Devanagari text cannot match below script-folded, so a report showing no source-exact matches between the Ṛgveda and the Sāmaveda is reporting an alphabet difference, not the absence of repetition. And the levels are explicitly not nested, a fact relied on in thirty-five places.

6.2 Exact parallels and variants

Figure 5 gives the counts for all four predicates by work pair. An *exact parallel* is equality on a named comparison surface, and the surface is recorded on the edge. The live graph holds 1,006 such edges, and their stored methods name the surface: script-folded (600), sandhi-insensitive, accent-insensitive (150). A *VARIANT_OF* edge (788) records a pair that matches at one level and differs at a stronger one — a real difference in transmission rather than a comparison artefact.

6.3 Near parallels: banded MinHash, then a composite score

Near parallels are the only layer where a genuine similarity threshold is applied, and the corpus makes the problem tractable in a way worth stating. Over the sandhi-insensitive surface, random-pair character-4-gram Jaccard has

- **At least 3 distinct mantras.** Two occurrences is as often a coincidence of sandhi as a formula.
- **At least 12 characters.** Two very short words fold to a handful of letters and match everywhere.
- **At most 8 % of a Veda.** A span occurring in more than that share is that Veda’s grammar, not its phraseology — and the share is computed *per Veda*, so a Sāmavedic refrain is judged against the Sāmaveda rather than against the corpus.
- **Ceilings** of 6,000 formula nodes and 120,000 occurrence edges, the first because the node exists precisely to replace combinatorial edges with a hub, and a hundred thousand hubs would defeat that.

Occurrence detection runs at two grains, and the edges record which: word-aligned matching (20,577 edges) and a sandhi-insensitive substring fallback (2,109). Family membership is likewise two mechanisms kept apart on the edge — a core grouping (1,828), expansion by containment (2,238), and a small similarity-based extension (8) — because the tier of a membership edge depends on which derivation produced it, and the two directions of the mirror must be byte-identical or a query would get a different tier depending on which way it walked.

Contamination is a named hazard rather than an assumed absence: a documented baseline finding records that formula nesting inflates roughly a third of formula-bearing passages, and the containment relation is modelled explicitly so that nesting is visible instead of double-counted.

6.6 Ritual and material culture

The ritual layer is small but it is the clearest case in the graph for typing at the edge rather than at the predicate, so we report it here rather than in an appendix.

The objects are modelled as distinct kinds rather than as one “thing” class: 103 rites, 3,121 ritual steps, 20 ritual roles, 21 offerings, 33 substances, 41 implements, and small populations of plants (22), animals (15), metals (7), weapons (5) and crops (5). The step layer is the large one, and it is source-explicit: a rite’s procedural steps are what a source prints, and all 3,121 carry that layer. An earlier pass had filed those steps under a generic “Other” category, which is a reminder that a fine-grained vocabulary is worth nothing if the loader does not use it.

The instructive part is the relationships. Consider two predicates:

Predicate	L2 deterministic-derived or L1 source-explicit	L4 interpretive-claim
USED_FOR_RITE	110	419
PERFORMED_BY	16	16

Each carries claims of two different epistemic kinds under one name. Some uses of a verse in a rite are derivable from what a source states about the book the verse sits in; others are this project’s reading of what the verse is for. Some statements of who performs a rite are source-stated; an equal number are inferred. A grade attached to the *predicate* would be wrong for one population in each row — which is exactly the defect [Section 4](#) describes, caught here before it shipped rather than after.

The layer also contains the graph’s only model-extracted ritual claims: 11 offering involvements, 9 ritual involvements and 3 substance involvements, 23 edges in total, all at L3 LLM-extracted and all excluded by a single predicate from any count a reader wants to keep source-grounded.

Two precision controls are worth naming. Rejected candidates are retained with reasons rather than deleted, so the offering layer ships alongside a file of refusals. And the layer’s own precision was sampled rather than assumed: a derived metric records two figures for the ritual-context assignment — 0.79 over every reviewed row and 0.95 over the reproducible ones — with a note explaining that a single figure would have to decide whether an unreproducible row is an error or an unknown, and that it is an unknown. That node also carries `is_human_gold = false` and a reviewer field naming an agent, which is how the graph says that its own quality figure is not human-adjudicated either.

6.7 Audio, briefly

Recitation audio is referenced rather than held: there is no recording node type in the graph, and the media are linked, not mirrored. We mention the layer only because its history contains the cleanest instance of a defect this paper’s method is meant to catch. An earlier source published one file per hymn and renumbered one R̥gvedic book after a different edition’s arrangement, so a key-for-key mapping attached the wrong recitation to 55 hymns — not a gap but a silent misalignment, which the project’s own decision record calls the worse of the two. The replacement source was verified per verse against the stored text, and the verses it could not verify were refused rather than mapped approximately. A separate owner decision then made human audible review a *badge* rather than a visibility gate, on the reasoning that automated checking is not listening; the number of recordings a named person has actually played remains small and is recorded as such.

6.8 Direction: compilation asymmetry

Identifying that two verses are parallel says nothing about which way the borrowing ran. The project initially asserted direction for exactly one pair — 1,684 Sāmaveda-to-Ṛgveda edges — on a *tradition*’s statement that the Ārcika is a collection of Ṛgvedic verses arranged for chanting, and left every other pair undirected on the ground that asserting a direction there would be a chronology claim. The weakness was that all 1,684 edges carried the identical justification sentence: a pipeline constant that says why the class is directed and nothing about the edge it sits on.

The replacement is a measured, per-edge signal. A borrowed verse and its source are by construction nearly identical, so nothing about the strings can point. What points is the company each verse keeps. A compiler assembles a unit out of verses drawn from elsewhere, so its unit is largely made of counterparts, and those counterparts are scattered across many units of the other corpus; a composer’s hymn is not made of anything, and only some of its verses happen to have been taken. For each side, measured on that side’s own containing unit:

$$\text{share} = \frac{|\{v \in U : v \text{ has a counterpart}\}|}{|U|}, \quad \text{dispersion} = |\{U' : \text{some counterpart lies in } U'\}|.$$

Direction is asserted from the high-share side to the low-share side, and only when share ≥ 0.5 and the margin between sides is ≥ 0.25 .

Two preconditions refuse pairs before the measure runs, both on measured grounds. *Ṛgvedic mediation*: if two verses in different corpora are both counterparts of the same Ṛgvedic verse, those corpora share Ṛgvedic text rather than each other’s — measured at 96.1 % of Atharvaveda/Sāmaveda pairs, 95.4 % of Sāmaveda/Yajurveda and 83.3 % of Atharvaveda/Yajurveda, against 0.0 % of Ṛgveda/Yajurveda. *Unit granularity*: the measure is a ratio over a structural unit, and the median containing unit holds 9 verses in the Ṛgveda, 4 in the Atharvaveda, 3 in the Sāmaveda — and 44 in the Yajurveda, whose smallest container here is the *adhyāya*. A share over a 44-verse unit is not the same instrument as a share over the three-verse unit that is modal in the Sāmaveda (281 of its 458 containing units hold exactly three verses, whatever the addressing calls them), so every Yajurvedic pair is refused.

Finally the instrument is only allowed to speak in a scope where it demonstrates both power (at least 100 directed verdicts) and internal consistency (at least 90 % pointing the same way). The scope table is the interesting output:

Scope	Directed	Consistency	Outcome
Ṛgveda/Sāmaveda, whole pair	1,212	0.991	accepted, SV $\rightarrow$ RV
Atharvaveda/Ṛgveda, whole pair	480	0.763	rejected
Atharvaveda/Ṛgveda, kāṇḍa 20	322	0.966	accepted, AV $\rightarrow$ RV
Ṛgveda/Yajurveda, whole pair	325	0.532	rejected

The method recovers the tradition’s own claim about the Sāmaveda independently, at 0.991; it localises the Atharvavedic case to kāṇḍa 20, which is exactly the book known to be largely Ṛgvedic; and it declines the two scopes where it cannot separate the sides. Four typed refusal codes record which precondition or floor rejected a pair.

6.9 What the corpus’s structure does *not* establish

One negative result belongs here because it disciplines the whole layer. The Sāmaveda has the highest undirected parallel coverage of the four works held here, at 1,677 of 1,844 verses — 90.9 %, against 34.8 % for the next-placed Yajurveda — and that figure is routinely read as self-evident confirmation that the Sāmaveda borrows. Against a degree-preserving permutation null over 400 draws, Ṛgveda/Sāmaveda containment is 6.69 $\times$ observed against 5.72 $\times$ expected from corpus size alone: an excess of only 1.17 $\times$. The recorded conclusion is that no size-independent structural signature in anything the project holds distinguishes borrower from source for any pair — which is precisely why direction had to come from compilation asymmetry and from a tradition’s explicit statement, rather than from coverage.

The same audit exposed a typing defect of the kind [Section 4](#) describes: the parallel edges carry an evidence surface of SANSKRIT, which is true of the pairing and false of the direction. The pairing is derived from the Sanskrit; the direction is not.

6.10 A refused layer

For completeness: a semantic-resemblance layer was proposed and refused before import, on a re-implemented lexical control it could not beat ([Section 8.5](#)). Zero assertions were written. Its absence from [Table 4](#) is a decision, not an omission.

Table 4. Cross-textual relationships between mantras, by predicate and work pair. The four predicates are deliberately not collapsed into one “similar to”: they assert different things, and the strongest, REUSES_TEXT_FROM, is directed only where a tradition states the direction (Section 6.8). Pairs are shown as source → target.

Predicate	Edges	By work pair
EXACT_PARALLEL_OF	1,006	AV→RV 551, RV→RV 256, RV→SV 89, SV→YV 70, RV→YV 24, AV→SV 9, AV→YV 7
NEAR_PARALLEL_OF	3,049	RV→SV 1,180, AV→RV 752, RV→YV 473, AV→SV 326, AV→YV 162, SV→YV 156
VARIANT_OF	788	RV→SV 415, RV→YV 165, AV→SV 132, AV→YV 41, AV→RV 22, SV→YV 13
REUSES_TEXT_FROM	1,995	SV→RV 1,684, AV→RV 311

Table 5. Sāmavedic notation: every one of the 1,844 Ārcika verses carries exactly one typed disposition, so a reader can never mistake a withheld verse for a verse the tradition does not notate. The third withheld class did not exist until an adversarial gate falsified the stated reasons of 39 rows.

Disposition	Verses	
notation present, source-explicit	1,136	the witness prints marks on this verse
withheld: near line, witness disagreement	565	needs philological adjudication
withheld: no near line, probable absence	104	the witness may not carry the verse
withheld: occurrence consumed by a repeat	39	added by Gate C (Section 7.1)
Total	1,844	verses with no disposition at all: 0

7 Case Studies

7.1 Sāmavedic notation: gating the withholding, not only the release

The Sāmaveda is the Veda of chant, and its Ārcika verses are printed in some witnesses with svara marks indicating the melodic treatment of each syllable. This layer is the project’s sharpest test of the evidence model, for three reasons: the marks are genuinely source-explicit, decoding them into pitch would be genuinely interpretive, and the coverage is genuinely partial.

What was admitted. 1,136 of the 1,844 Ārcika verses carry an accented witness (Table 5, Fig. 6), imported as a second TextVersion of the verse’s own text in the role PARALLEL_TEXT, never replacing the primary text. We verified all of this against the live store: 1,136 accented parallel text versions against 1,844 primary ones, and 25,542 tone marks on the accented rows. Coverage is published as four figures rather than one — CHANDA 471/585, ARANYA 31/55, MAHANAMNYA 3/10, UTTARA 631/1194 — because a single total of 1,136 hides which part of the Ārcika a reader can and cannot see notation for.

What was refused, and how the refusal is enforced. The marks are recorded as data and never decoded. The Kauthuma decipherment authority is not held, and a study of a different school’s chant was deliberately not applied. The refusal is enforced at four levels rather than stated once: the row schema pins the payload to a closed set of thirteen field names and checks that none of them names a pitch; the predicate is the ordinary HAS_TEXT_VERSION, and a melodic predicate is verified absent from the store’s type table; the migration plan declares six explicit refusals; and a live test scans every property key on the witness for pitch vocabulary, permitting exactly one match — the property that *records* the refusal. We confirmed the outcome: no melodic relationship type exists in the graph, and the property notation_is_interpreted_into_pitch is present on all 1,136 witnesses with the value *false*. A first version of that scan flagged the guard as the thing it guards against, which is a small but exact illustration of how hard it is to write a check that means what it says.

Gate C, and the negative control. The layer was audited by a gate implemented separately from the pipeline that produced it, and the separation is explicit: Gate C names the fields it refuses to read — including the pipeline’s own tone-stripped text and its quality report — and re-implements the declared normalisation from the manifest’s prose, using Unicode character categories rather than the artefact’s integer ranges. Eight attacks ran at full coverage over 6,232 row-evaluations.

The design feature we want to draw out is the second attack. Having re-derived each verse’s canonical text and found it aligned on 1,136 of 1,136, the gate then ran the *same* fold against the *next* verse, and got 0 of 1,136. The gate’s own statement is the methodological point: the aligned result is only evidence to the extent that this one is near zero. An agreement measurement without a negative control does not distinguish a working method from a trivially satisfiable one.

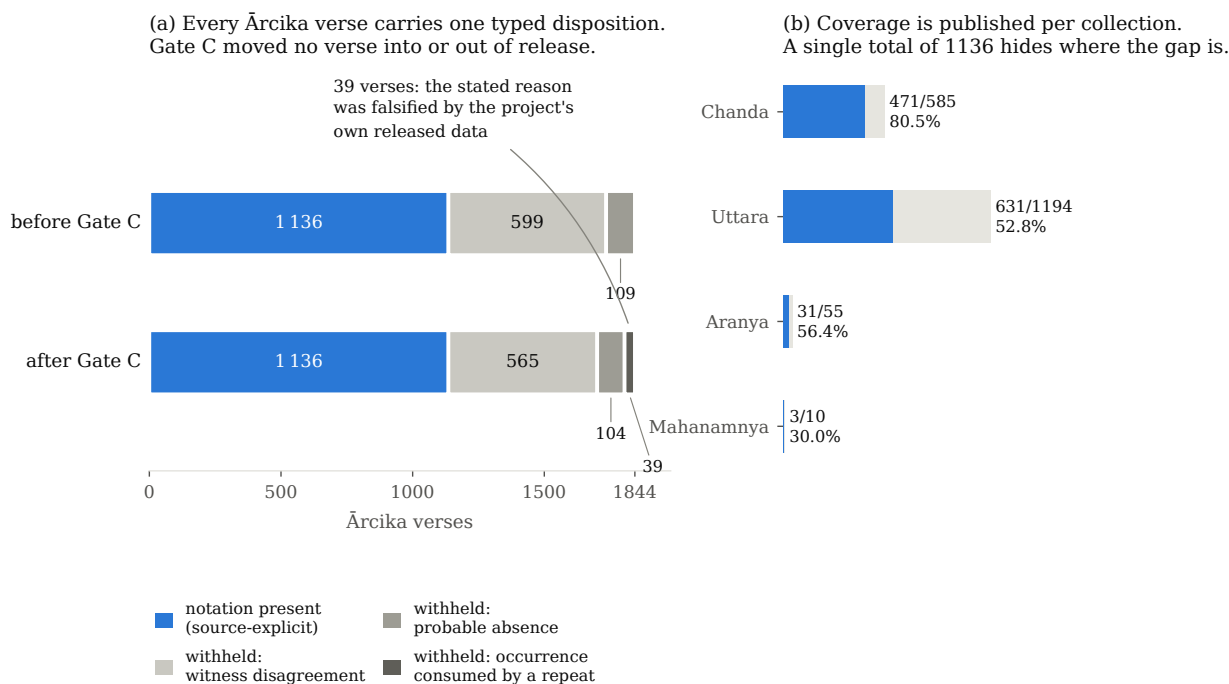


Figure 6. The Sāmavedic notation layer before and after the adversarial gate. (a) Every one of the 1,844 Ārcika verses carries exactly one typed disposition. Gate C moved no verse into or out of release; it added a third withheld class and moved 39 verses into it, because the reasons those rows stated were falsified by the project’s own released data. (b) Coverage is published per collection rather than as one total, because a total of 1,136 hides which part of the Ārcika a reader can and cannot see notation for.

The finding: 39 withheld rows whose reason the repository disproved. Seven attacks survived. The seventh did not, and it is the one that attacked the project’s own withholdings. 708 verses were withheld under two declared reason classes: 599 saying a near line exists so the difference needs philological adjudication, and 109 saying the accented witness plausibly does not carry the verse at all. The attack observed that both claims are falsifiable from the repository itself — if a withheld verse’s exact normalised text is *released*, with notation, on a coreferent twin elsewhere in the Ārcika, then the witness demonstrably carries the text and neither stated reason is the true mechanism.

Thirty-nine verses failed this test: 34 from the witness-disagreement class and 5 from the probable-absence class, each with at least one released twin carrying byte-identical normalised text. The true mechanism was then measured rather than guessed: of 202 repeated-text groups, 153 have every member released, so the witness does print a repeated verse more than once and the harvest does find both lines. Where only one member is released, the single order-consistent occurrence was consumed by the earlier verse and the later one had no line left. That is line exhaustion on a coreferent repeat — neither an absence from the witness nor a philological disagreement.

What changed, and what deliberately did not. A third withheld class was declared, `WITNESS_OCCURRENCE_CONSUMED_BY_A_COREFERENT_REPEAT`, and the 39 rows were moved into it; we read the corrected distribution off the live graph as 565 / 104 / 39, summing to the unchanged 708. *No verse moved from withheld to released, and no notation value was invented.* The gate’s reasoning is that the notated string belongs to the coordinate the witness printed it at, so copying a twin’s marks onto another verse would be inventing notation — only the stated reason changes. The original reason text is preserved verbatim on each corrected row, and because the staging directory is a checksum-sealed artefact whose generator is not in the repository, the correction was applied as an overlay rather than an edit, leaving all twelve manifest checksums valid.

Finally, the failing run is preserved on disk and a test asserts that it still records 39 defects, so the finding cannot be erased by regenerating only the passing run. Gate C also declined to count the artefact’s own pre-existing adversarial sample, on the ground that it targets a different population and was run by the same pass that produced it.

Why the gate existed at all. An owner decision had released the notation layer from an unrelated audio gate on four conditions, three of which measured met. The fourth required that “existing validation gates pass” — and the eligibility record showed one gate `UNKNOWN` and the other `NOT_RUN`. The condition was recorded as unmet, with the reasoning that reading “gates” as the single gate that already existed would clear the board by narrowing a

Table 6. Translation coverage as a typed state rather than a boolean. Each of the 20,210 mantras carries exactly one terminal state, so an absence always arrives with its reason attached. The two UNCOVERED states are distinct on purpose: one says a translated identical parallel exists elsewhere in the corpus, the other says nothing reaches the verse at all. The Sāmavedic column is the reason the distinction was built.

Verse coverage state	RV	AV	YV	SV	Total
DEDICATED_TRANSLATION	10,322	5,715	1,950	0	17,987
its own 1:1 English rendering					
RANGE_TRANSLATION_ANCHOR	30	34	0	0	64
anchors a print unit spanning it and others					
RANGE_COVERED	36	52	0	0	88
carries no edge; named in another verse’s span					
CONTAINER_TRANSLATION	158	0	0	0	158
only rendering is aligned to a hymn					
REUSED_RENDERING	0	21	0	173	194
another corpus’s rendering of identical Sanskrit					
UNCOVERED_REUSABLE_PARALLEL_AVAILABLE	0	6	0	1,488	1,494
nothing reaches it, but a translated identical parallel exists					
UNCOVERED_NO_RENDERING_REACHES_IT	6	11	25	183	225
nothing reaches it, and no parallel does					
Total	10,552	5,839	1,975	1,844	20,210

word, and that the conservative reading is the one that cannot manufacture a closure. A second owner ruling confirmed the conservative reading, so the two gates were built rather than the word narrowed. The newly built gate immediately landed a real defect.

7.2 Translation: coverage is a kind, not a boolean

Three separate mechanisms in this project once answered the question “is this verse translated?” and all three answered it by testing for a translation edge. That test was correct while every translation was a one-to-one rendering of its own verse, and it stopped being correct the moment anything else was admitted.

Why alignment is hard here. Five failure modes were measured, and none is a data-quality problem that better sources would fix. The editions do not share a verse spine: for a run of *dvipada virāj* hymns the canonical verses are single hemistichs while the translator numbers the four-pāda group as one unit, printing five units against ten verses. The available translation may be of a *different recension*: the standard English Sāmaveda is Rāṇāyāniya on its own translator’s preface, while this corpus is Kauthuma, and nine avenues were searched without finding a public Kauthuma Ārcika translation. Position cannot substitute, because the offset between the two orders drifts monotonically from 0 to –81. The source’s own printed labels are sometimes wrong — one book prints the sequence 23, 24, 23, 26, a misprint for 25. The corpus repeats itself, so “same English, different verse” is normal rather than a defect; two of the packet’s own duplication tests were wrong for exactly that reason before the third was right. And a translation can be present, correctly typed, and still invisible to a reader’s edge test, which is how a range alignment went unseen.

The typed model. The replacement is two closed vocabularies and one refusal. A *coverage kind* says what a translation gives a verse — dedicated, range, container, or reused — and is explicitly not rankable; the classifier *raises* on an unrecognised alignment level rather than defaulting, because defaulting is how a fourth kind would silently be reported as a verse’s own translation. A *verse coverage state* says what a verse has, in eight values of which three are negative and distinguish genuinely different situations (Table 6). Alignment confidence is a six-valued typed vocabulary rather than a number pretending to be a probability; the largest class, covering 17,095 of 18,427 renderings, is MACHINE_ALIGNED_NO_PER_ROW_CONTROL, which names its own weakness.

What the Sāmavedic column shows. 1,488 Sāmavedic verses are uncovered *but* have a translated identical parallel elsewhere in the corpus; 183 are uncovered with no parallel at all; and 173 carry a reused R̥gvedic rendering admitted only on Sanskrit verified character-identical. The last of these is the case the model was built for. Those 173 are visible to a reader, and they are counted in no Sāmavedic translation figure: the code holds two frozen sets,

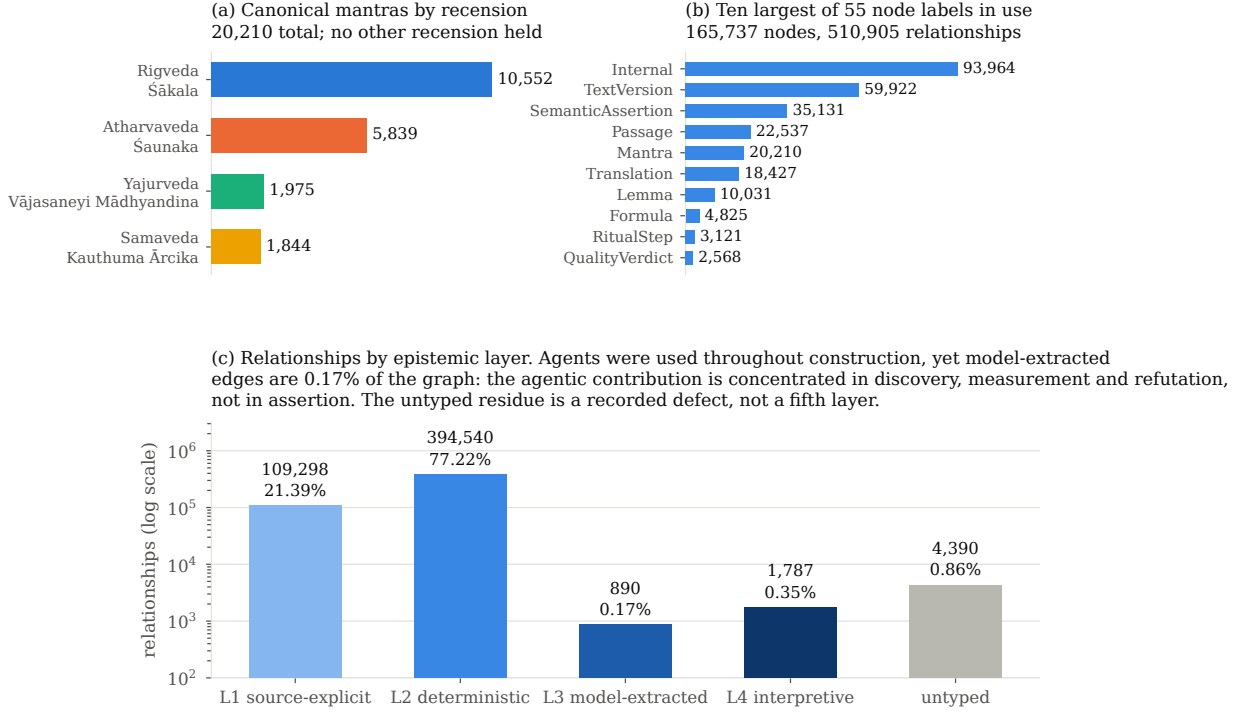


Figure 7. Corpus, graph scale, and the distribution of epistemic types. **(a)** The canonical corpus is four named recensions, not “the Vedas”. **(b)** The ten largest node labels; Internal is a marker on non-public objects, not a domain type. **(c)** The paper’s central measurement, on a log scale: agents were used continuously throughout construction, and model-extracted relationships are 0.17 % of the graph. The untyped residue is a recorded defect in the grading contract, not a fifth layer. Measured on the live store at commit 6bf220c.

one for “English reaches the reader” and one for “independent English exists”, differing by exactly the reused class, and records that a total reporting 173 English translations for the Sāmaveda is wrong — a mistake an earlier draft of the staging made before the reuse column was checked. The independent-English count for the Sāmaveda is zero.

Two disciplines worth naming. First, the product shipped *before* the data: the import plan was built and hashed but deliberately not executed in the commit that introduced the typed model, on the reasoning that the interface must be able to represent a range and a reused rendering before such rows land, not afterwards. Second, the campaign restated one of its own headline figures *downward* on purpose. Narrowing “translated” to mean a verse’s own rendering moved the R̥gvedic figure from 10,502 to 10,479, and a test now asserts the range population beside it so that the narrowing cannot pass as a regression.

And a defect this found. A prior import had attached 25 renderings to the wrong verses, precisely because two editions did not share a verse spine and the guard reported the mismatch as “missing” rather than as a spine divergence. The rows were re-anchored and the readback was clean — but the more useful artefact is the reason code introduced alongside: `SOURCE_EDITION_VERSE_DIVISION_DIFFERS`, which is now one of six typed rejection reasons rather than an unexplained absence.

8 Evaluation

Evaluation of a construction method like this one is awkward, because the thing to be evaluated is not a single model’s output but a discipline. We report five kinds of measurement: the graph’s scale and internal integrity, the distribution of epistemic types, precision against an external reference set, the outcomes of adversarial gates, and one downstream question-answering benchmark. We then state at length what none of these establish.

8.1 Scale and integrity

At the frozen commit the graph holds 165,737 nodes and 510,905 relationships across 55 node labels in use and 81 relationship types (Fig. 7, Table 7). Of the nodes, 71,773 are public and 93,964 carry the internal marker. Every one of the 71,773 public nodes has a non-null public identifier and the identifiers are distinct.

Table 7. The fourteen largest node labels and relationship types, of 55 labels in use and 81 relationship types. A further label, `SourceArtifact`, is declared in the store’s token table but carries no nodes.

Node label	Nodes	Relationship type	Edges
Internal	93,964	MENTIONS_LEMMA	154,261
TextVersion	59,922	HAS_TEXT_VERSION	59,922
SemanticAssertion	35,131	HAS_SEMANTIC_ASSERTION	35,131
Passage	22,537	ASSERTION_PREDICATE	32,938
Mantra	20,210	MENTIONS_ENTITY	28,122
Translation	18,427	ABOUT_CONCEPT	24,861
Lemma	10,031	USES_FORMULA	22,686
Formula	4,825	CONTAINS	22,537
RitualStep	3,121	HAS_TRANSLATION	18,427
QualityVerdict	2,568	HAS_RISHI	17,889
RoleFiller	2,052	MENTIONS_DEVATA	17,165
DerivedMetric	1,485	HAS_CHANDAS	16,298
PadaParallelGroup	1,434	HAS_DEVATA	10,558
QAIssue	915	SHARES_FORMULA_WITH	6,148

Integrity is gated rather than asserted, and we ran the gates rather than quoting the project’s certification of them. That distinction turned out to matter.

The eleven scorecard gates all read zero: internal leakage, ungraded edges, nodes without a label, orphan entities, signature violations, undeclared relationship types, interpretive claims graded other than the weakest tier, claims and metrics pointing at passages, orphan public nodes, and labels falsely declared unpopulated.

The twenty live invariants do not all pass. Running the suite at the frozen commit gives 17 ok, 1 warning and 2 failures:

Severity	Invariant	Edges
FAIL	edges_without_grade_basis	165,788
FAIL	edges_without_attribution_precision	19,598
WARN	assertions_without_predicate_edge	2,193

So 32.5 % of edges cannot say *why* they carry the grade they carry, and 3.8 % do not say whether they are a perverse claim, an inherited label, a textual mention, or not an attribution at all. Both invariants are declared at failing severity by the repository itself.

We report this prominently because of how we came to it. An earlier draft of this paper asserted that all twenty invariants passed, on the authority of the project’s release certification — which recorded that state truthfully at an earlier census. We had not run the suite. An adversarial reviewer of this manuscript ran it in one command and the claim fell. That is precisely the failure the paper is about: a report was trusted where a measurement was available, by the authors of a paper arguing that reports are not evidence. It is recorded in `supplementary/peer_review.md` and we have not smoothed it away here.

What does hold as a universal quantifier, verified: no edge without a quality tier (510,905 of 510,905), no unlabelled node, no product node reaching a diagnostic node, no attribution-precision value outside the declared enumeration, no work without a scope statement, and no predicate carrying more than one run identifier.

One coverage gap we report rather than round away. The quality tier is present on 100.00 % of edges. The *knowledge layer* is present on 99.14 %: 4,390 edges carry no layer, and they are not scattered — they are 4,079 `HAS_PARALLEL_PADA` edges and 311 `REUSES_TEXT_FROM` edges. Worse than their number is the grade they carry: *all 4,390 are TIER_B*, the tier the model defines as “a reproducible rule derived it”. An edge whose origin the graph does not record is therefore presented to a reader as reproducibly derived. The tier-from-layer mapping is also not the total bijection [Section 4](#) describes: besides the documented candidate rule (292 model-extracted edges at `TIER_D`), 2,469 deterministically derived edges carry `TIER_D` and 36 source-explicit edges carry `TIER_B`. This is a real residual defect in the graph-wide typing contract, not a fifth layer, and the percentages in [Table 2](#) are computed over all 510,905 edges rather than over the typed subset, so the gap is visible rather than divided away.

8.2 The distribution of epistemic types

The central measurement of this paper is [Table 2](#). Model-extracted relationships number 890, or 0.17 % of the graph; interpretive claims number 1,787, or 0.35 %. The remaining 98.6 % is either what a source states (21.39 %) or what a reproducible rule derives from what a source states (77.22 %).

The ratio depends on the population, so we give four. A reviewer of this manuscript observed, correctly, that 0.17 % is the smallest of several defensible figures and that quoting only it overstates the result. The denominator of 510,905 includes populations no extraction method could ever author — 154,261 lemma mentions, 59,922 text-version links, 22,537 containment edges and 18,427 translation links, 255,147 in all. Over the 255,758 edges that remain once that plumbing is removed, the model-extracted share is 0.35 %. Over the semantic assertion layer alone — the population where a model is actually the proposing mechanism — it is 2,459 of 35,131, or **7.0 %**. And if one counts every edge whose existence is owed to a model proposal rather than only those carrying the layer, the figure is higher again.

All four are true of different questions. The one the paper’s argument needs is not the smallest but the one over the population at risk: **where a model could assert, it authored 7.0 % of the assertions, and none of them was promoted**. We keep 0.17 % as the graph-wide figure because a consumer filtering the whole store is filtering the whole store, and we state the others here so that the choice of denominator is the reader’s rather than ours.

At node level the same shape holds: of 165,737 nodes, 2,459 carry the model-extracted layer, all of them `SemanticAssertion` nodes with derivation `MODEL_EXTRACTION`. The remaining 32,672 semantic assertions are derived by morphological rule over a published scholarly annotation or from a treebank dependency relation, and the project’s own code refuses to blend the two totals, on the ground that a blended figure states a confidence about all 35,131 assertions that is true of neither population.

And 89 % of the model layer is not wired. Of the 2,459 model-extracted assertions, **2,193 carry no `ASSERTION_PREDICATE` edge at all** — their predicate is a property but not a relationship, so predicate-level traversal cannot see them. The repository’s own invariant suite reports this at warning severity, and it is the third adversarial finding from an earlier round that was never remediated. It bears directly on the figure above: had those assertions been wired as the deterministic derivations are, the model-extracted edge count would be 890 plus 2,193, and the graph-wide share would be roughly 0.6 % rather than 0.17 %. Part of what the headline ratio measures is a wiring defect rather than a discipline.

Two further measurements characterise how much the semantic layer withholds. Of the 35,131 assertions, 28,734 carry `roles_withheld`, and only 341 are three-slot complete — that is, fewer than 1 % of the semantic assertions assert a full agent–predicate–patient frame. The layer is overwhelmingly a predicate layer with the role slots deliberately empty ([Section 9](#), F2).

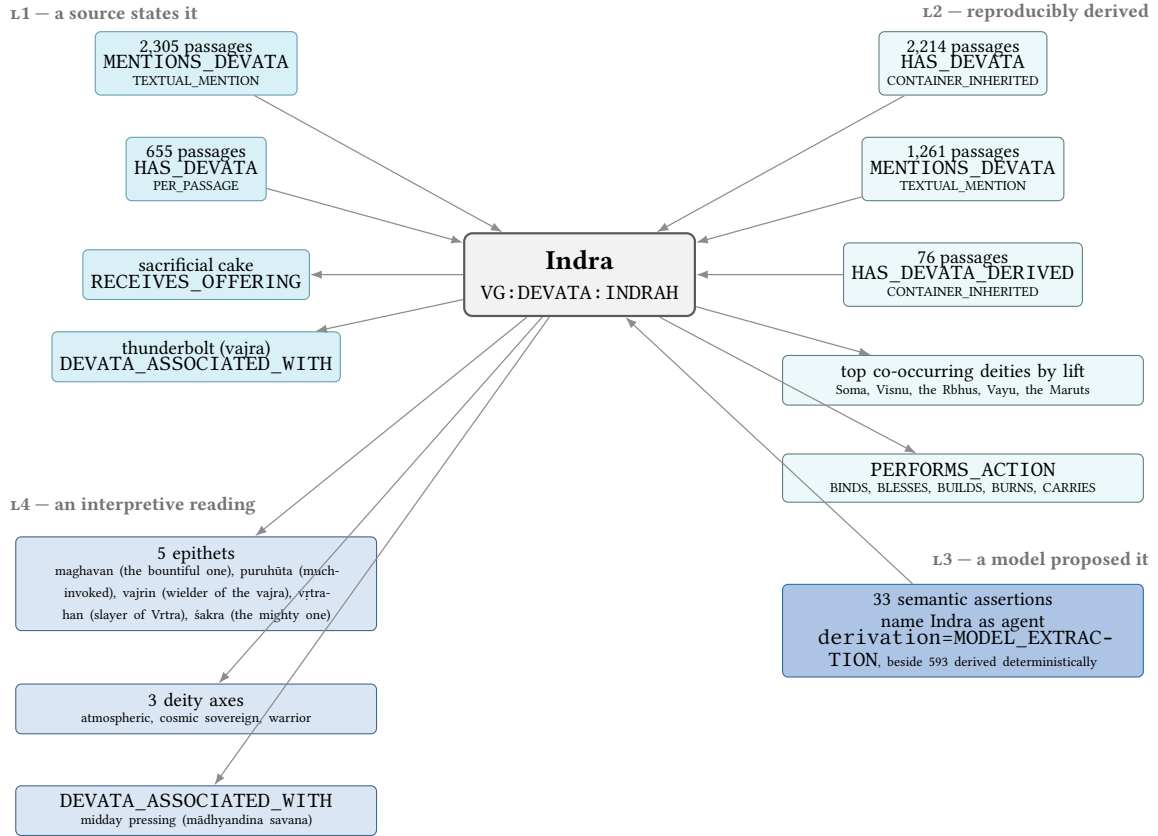
8.3 Precision against an external reference set

No human gold standard exists for any VedGraph claim. We use the project’s own term, `INDEPENDENT_SOURCE_ADJUDICATED_REFERENCE_SET`, for what does exist, and we use it strictly.

What it is. VedGraph’s lexical and mention claims were machine-compared against the Treebank of Vedic Sanskrit ([Hellwig et al. 2020](#)), a published, human-validated annotation of 27,182 sentences. Verse addresses were established by de-accented, de-spaced skeleton containment inside the cited hymn with a similarity floor of 0.62 and a requirement that the winner beat the runner-up by 0.06; the alignment was then checked for monotonicity (5,541 adjacent pairs, 90 backwards jumps at 1.62 %, with both sides of every jump quarantined and 176 sentences dropped) and confirmed against an independent lemma annotation already in the graph. The resulting set is 3,951 rows folding 11,210 adjudicated claims, with 805 rows refused for an ambiguous margin and 77 below the floor. It reaches 3,072 of the 20,210 mantras.

What it measures. On the lexical layers the reference set gives precision in the range 0.982–0.994 with recall 0.443–0.543. The precision figure is the useful one; the recall figure is bounded by the treebank’s own coverage rather than by the layer’s behaviour.

What it is not. The project’s own statement is that the human annotation is of the *Vedic text*, not of VedGraph’s claims, and that the verdict was derived from it mechanically — one step removed from human review of the project’s own assertions, and the strongest thing obtainable without commissioning an annotator. The reference set cannot decide aboutness or referent. It reaches zero Sāmavedic mantras, so the 11,380 enrichment edges on



The split inside HAS_DEVATA is the point: the same predicate reaches this deity both as a per-passage statement and as a label inherited from the enclosing hymn, and only *attribution_precision* tells them apart. A reader who counts “Indra’s verses” from this predicate alone reports 2,869 where the source states 655.

Figure 8. One entity, four epistemic layers: the neighbourhood of Indra, queried live. High-cardinality passage predicates are aggregated by (predicate, layer, attribution precision); low-cardinality typed neighbours are drawn individually and capped. Selection criteria are stated in the generating script. The split inside HAS_DEVATA is the point of the figure.

the Sāmaveda — every outgoing edge from an Ārcika mantra other than text, translation and containment — are not merely unannotated but, in the project’s word, unscorable.

Zero human annotation, verified. Across 695 rows in the gold directory there are no human annotations. The 120-row semantic scaffold is empty — every row UNANNOTATED, zero entities and zero relations. The 575-row theonym set is labelled MODEL_ADJUDICATED by a single model. We confirmed the consequence in the live store: no edge carries a human-reviewed provenance on any of four candidate properties, the only node bearing an *is_human_gold* property carries the value *false*, and the strongest review state present is MODEL_ADJUDICATED on 613 edges. The 1,166 quality-verdict nodes each carry an explicit *not_human_gold_because* property.

The single most informative number here is a negative one. Of 74 theonym rows the treebank could reach, 72 were decidable, and the model adjudication agreed with the human annotation on 54 — 75 %. A quarter divergence on the only independent test available is the reason the set is not promoted to gold, and it is the best available estimate of what single-model adjudication is worth in this domain.

Agent agreement is reported, and is never called accuracy. Inter-run and inter-agent agreement figures exist and are labelled as such: a 12-replica cross-agent calibration reaching a mean Jaccard of 0.832 on assertion sets but exact set agreement on only 1 of 12; a two-run stability check whose verdict is recorded as POLICY_STABILITY_ONLY_NOT_ACCURACY; and a pair of earlier model runs disagreeing on 91.6 % of 120 mantras. The only Cohen’s κ in the repository (0.567 four-way, 0.366 binary) is model against model. Every such report carries the project’s standing banner that model self-agreement is not accuracy.

8.4 Adversarial gate outcomes

Fourteen domains passed through a three-gate structure (Section 5.6). The outcomes worth reporting are the failures, since a gate that never fails is not evidence. Across the campaign, gates rejected or forced the withdrawal of: 13,187 elements that the domain rules said never to import; 5,930 edges over-written by a first import attempt; 12,033 nodes left unreachable by a second; 102 curated display labels silently overwritten by a third; 33 metre assertions whose object was not a metre; 49,179 of 51,231 semantic role fillers; and the stated reasons of 39 withheld Sāmavedic verses (Section 7.1).

Three structural features of the regime are worth separating from its results. First, several gates were built only after an owner declined to narrow the word “gates” to mean the one gate that already existed; the newly built gate then landed a real defect on its first run. Second, a failing run is preserved on disk and pinned by a test asserting its defect count, so a finding cannot be erased by regenerating only the passing run. Third, at least one gate was found to be incapable of failing: it computed its set of declared relationship types as the union of the ontology and the graph, then asked which graph types were missing — and so reported zero undeclared types for an entire wave while eleven predicates went unclassified.

8.5 A baseline that the project’s own semantic layer lost to

One genuine controlled comparison exists. A semantic-resemblance layer was proposed, and before importing it the project re-implemented a purely lexical control — a character-4-gram similarity — and evaluated both against 279 adjudicated pairs. The lexical control reached an AUC of 0.754, and no semantic candidate beat it. The layer was refused: zero assertions imported, the graph unmutated, the outcome recorded as VERIFIED_ZERO /REFUSED.

Two details make this more useful than a bare negative result. The prior run’s reported control figure did not reproduce — 6 of 370 pairs matched exactly — and the project’s response was to withdraw the carry-over argument entirely rather than reconcile the numbers, recording that no prior number travels forward. And the adjudication labels for those 279 pairs are themselves LLM-produced, with 162 of 486 pairs left UNADJUDICATED_QUOTA_EXHAUSTED; the comparison is therefore between two candidate generators judged by a third model, which is what makes it a usable relative ranking and not an absolute precision figure.

8.6 Downstream: a question-answering benchmark

The graph supports a retrieval-and-synthesis interface, and a 60-question benchmark evaluates it. We report it as downstream evidence that the evidence typing survives into use, not as a language-model evaluation.

The question set is frozen and hash-pinned, hand-authored in three cohorts (18 demonstration, 10 adversarial including two prompt-injection probes, 32 closure) over eight categories including four deliberately unanswerable questions. The freezing policy is explicit: questions are not edited in response to a poor answer, because a benchmark that moves when the model stumbles measures nothing.

Grading is hybrid and contains no LLM judge. The mechanical half is deterministic — all 60 evidence packets replayed from the live store to identical item counts, 304 citations audited, every integer in the prose matched against the packet’s numeric facts, Sanskrit runs classified for provenance, eight regex safety probes — and the composition refuses to run if a hard gate fails. The adjudicated half is a human reading each answer against its replayed packet.

The final state is 39 SUPPORTED_CORRECT, 4 PARTIAL_CORRECT, 17 INSUFFICIENT_EVIDENCE_CORRECTLY_REFUSED, 0 MISLEADING and 0 HALLUCINATED. Seventeen refusals out of sixty is the number we would draw a reader’s attention to: the interface declines more than a quarter of the questions put to it, which is the behaviour the evidence typing is for.

The caveats are severe and are the project’s own: a single provider and model, one sample per question, no repeated sampling, no cross-provider comparison, a single unnamed adjudicator, and therefore no inter-annotator agreement. Three answers were truncated by an output-token cap. The Sanskrit quote auditor over-reports, flagging 59 runs across 23 answers with zero fabrications among them, so readers of those 23 answers are warned away from wording that was never presented as a quotation.

8.7 Reproducibility

Reproducibility was measured rather than asserted, by re-running. The canonical corpus builds are byte-reproducible: rebuilding produces identical artefacts, and the identity scheme reproduces all 22,537 passage identifiers exactly (Section 3.3). A separate check verified that two independent source builds produce a public-identifier key set identical to the live graph’s.

We state the limit plainly, because the repository does. **A fresh clone cannot bootstrap a certified graph.** The corpus artefacts, registries, staging evidence, gates and build scripts are in version control; the loaded Neo4j

Table 8. Failure ledger. Each row is a case in which the project’s own output was withdrawn, refused or corrected. “Detected by” names the mechanism that caught it, which is the column that matters: in no case was the defect found by the process that produced the output.

	What was withdrawn or refused	Detected by	Rule that followed
F1	41,796 relationships deleted in one revision; 30,539 retired in the next, including 21,246 concept edges resting on one English keyword	a question no stored field could answer	if two populations need different trust and no field separates them, the field is missing
F2	49,179 of 51,231 semantic role fillers	sampling: 24.8 % wrong, unresponsive to tuning	an error rate that will not tune is an unsupported layer, not a miscalibrated one
F3	an entire image-transcription corpus; then the refusal’s own “monotone improvement” claim	inter-reader agreement 0.505 over 204 units; then re-reading the series	a refusal is a claim and gets the same scrutiny as an assertion
F4	a deity-community partition, then the decisive argument <i>for</i> refusing it	a second pass reading the project’s own curation notes	do not import the standard scholarly view as the graph’s model and then grade the graph against it
F5	a release candidate, with all 11 scorecard gates green	a registry with 84 of 85 entries open and no closed state expressible	a gate measures a property; completeness is a property of a registry
F6	nothing — a <i>correct</i> restoration of 30,266 nodes degraded the product	a legibility measurement at a fixed display budget	completeness and legibility are different objectives
F7	the stated reasons of 39 withheld Sāmavedic verses	a gate built to attack the withholding, with a negative control	gate the refusals, not only the assertions
F8	a semantic-resemblance layer, before import: 0 assertions written	a re-implemented lexical control at AUC 0.754 that nothing beat	re-implement the baseline rather than carrying its number forward
F9	33 metre assertions whose object was a bracket fragment, not a metre	a closed-world check on the object vocabulary	resolving into a contaminated vocabulary is not resolution
F10	13,187 imported elements the domain rules excluded; 5,930 edges over-written; 12,033 nodes orphaned; 102 curated labels overwritten	readback comparing rows <i>sent</i> against rows <i>landed</i>	an importer’s own success report is not evidence

store is not, several source artefacts cannot be redistributed under their recorded rights, and the generator of at least one sealed staging artefact is not in the repository at all — which is why a defect in it had to be corrected by an overlay rather than by editing the artefact. [Section 13](#) states which parts are available on what terms.

9 Failure Analysis and Threats to Validity

9.1 Failure-driven refinement

The project’s most useful output, for the purposes of this paper, is its record of being wrong. We report the cases in a uniform shape — what was produced, what detected the defect, why the first method failed, what changed, and the methodological rule that followed — because the rule is the transferable part. [Table 8](#) summarises; the text treats the six most instructive.

F1. A layer deleted 41,796 of its own edges, and its successor retired 30,539 more. The second revision of the knowledge model deleted 41,796 relationships that the same revision had created in error. The third then withdrew 30,539 further edges, of which 21,246 were concept-attribution edges that had fired on a single keyword in Griffith’s 1896 English. The detection was not a test but a question: what surface was this evidence read off? Because the answer was not recorded anywhere, the edges were indistinguishable from edges matched on the Sanskrit, and they carried an identical grade. *Rule:* if two populations require different trust and no stored field separates them, the field is missing, and its absence is a defect of the same order as a wrong value. The evidence-surface axis ([Section 4](#)) exists because of this case.

F2. Ninety-six per cent of the role layer withdrawn. A semantic role layer had asserted 51,231 role fillers. Sampling put the wrong-filler rate at 24.8 %. Repair was attempted first and failed on measurement: widening the gate moved the error only to 24.3 % at a cost of eighteen recall points, with per-role residue ranging from 11.4 % on goal to 31.2 % on location. The layer was reduced to the 2,052 fillers resting on a published treebank’s own

Table 9. The gap registry at 6bf220c. Until a late campaign the registry had no closed state at all, so every gate could be green while completeness was unresolvable (Section 9); the closure vocabulary below was added to make termination expressible.

Status	Entries
CLOSED_DERIVED	53
CLOSED_SCOPE_DECISION	14
BLOCKED_EXTERNAL_SOURCE_UNAVAILABLE	7
CLOSED_VERIFIED_ZERO	5
NEEDS_AUDIBLE_REVIEW	3
CLOSED_NOT_APPLICABLE	2
CLOSED_SOURCE_ACQUIRED	1
Total	85
of which closed by some route: 75	

labels; 28,734 assertions were stamped `roles_withheld` with a reason, and 15,704 withdrawn rows were staged as a verification queue rather than deleted. The project’s summary is the finding: the R̥gveda now has a predicate layer and no role layer at all, because it has no parse. *Rule*: when a measured error rate does not respond to tuning, the layer is not miscalibrated but unsupported, and withdrawal is the correct repair.

F3. A corpus refused on measured fidelity, then the refusal’s own reasoning corrected. An independent image transcription of an 1856 Atharvavedic edition was built and then refused: exact agreement between two independent readings of the same material was 0.505 over 204 units. What makes the case instructive is what happened to the *explanation*. A report had described the agreement series as a monotone climb that had “risen every time”; the actual series was 0.00, 0.26, 0.485, 0.505, 0.542, 0.522, and it later fell again to 0.500 over 272 units. The trend claim was withdrawn with the observation that asserting a trend from five points is the same overgeneralisation the paragraph was written to correct, committed inside the correction itself. A second claim — that the accent layer was unrecoverable — was also withdrawn, because one worker had captured it with a reproducible method; the gap was a convention divergence, not a capability limit. *Rule*: a refusal is a claim and needs the same scrutiny as an assertion.

F4. A refusal whose decisive argument was wrong. A deity-community partition was refused on the ground that every method separated two names for one deity. Re-examination overturned the refusal’s reasoning rather than the refusal: the project’s registry *deliberately* holds those two names apart, with high-confidence curation notes and no variant edge between them. The agent had imported the standard scholarly view as the graph’s model and then graded the partition defective for respecting the project’s own curation. The refusal was reformalised on a different, measurable basis and the original claim was recorded as withdrawn in the required form — original claim, withdrawal, reason. A related figure was corrected in both directions at once: an imbalance is tenfold over all 214 deities and 7.9-fold over the 157 eligible ones, peaking in different books. *Rule*: neither figure is wrong; they differ by population, so a population must be named beside any ratio.

F5. Every gate green, and the release still refused. At the close of a completeness campaign the scorecard read 11/11, dependency state was current, and every test suite passed — and the verdict was `DATA_COMPLETENESS_NOT_COMPLETE`, release candidate *no*. The cause was that the gap registry had no closed state: 84 of 85 entries were open, there was no closure vocabulary, and no field could hold one. The decisive individual finding was a registry entry recording “closed via the Wayback Machine, 944 of 961” against a graph holding exactly the pre-closure figure, with all 2,254 accepted staging rows targeting real mantras and none carrying a translation. *Rule*: a gate measures a property; completeness is a property of a *registry*, and a registry with no terminal state cannot express it. The closure vocabulary in Table 9 was added in response.

F6. The completeness campaign made the product worse. A correct restoration of 30,266 previously dropped semantic-assertion nodes doubled the size of the exported interactive artefact and pushed the share of the public graph that is apparatus — evidence records and passages rather than entities — to roughly 81 %. Measured at a display budget of 40 neighbours, the entry for Indra spent 21 seats on passages and evidence records and 6 on other deities. Nothing about the curation had changed; the corpus had. *Rule*: completeness and legibility are different objectives, and a graph that improves on the first can regress on the second without any individual claim becoming false.

9.2 What the adversarial rounds found in each other

Five hostile rounds ran over the release candidate. The pattern worth reporting is that later rounds found defects in earlier rounds’ own work: one round closed two critical findings that the same round’s hostile pass had raised against itself. A related case is the public-identity collision, in which 30,266 semantic-assertion nodes carried no assertion identifier and fell through to their source passage’s canonical key — collapsing two public objects onto one and silently dropping assertion-to-passage edges as self-edges.

The generalisable observation is that an agent auditing its own output finds less than a differently-instructed agent auditing the same output, and that the gap is large enough to matter at this scale. We are careful not to overstate this: the agents are not independent in the statistical sense. They share a model family, and in several cases the same family produced both the artefact and its audit.

9.3 Threats to validity

Construct: recension scope. Every absence measured here is an absence from four named recensions. The Kṛṣṇa Yajurveda, the Paippalāda Atharvaveda, all four Sāmavedic *gāna* collections and every post-Saṃhitā layer are absent. A reader who reads “the Atharvaveda does not say X” as a claim about the Atharvavedic tradition will be wrong.

Construct: the Atharvavedic corpus is a working corpus. The Atharvavedic text is held as a reference-only, working-private digital corpus carrying an explicit assertion that its lineage is *not* an independent transcription of the printed edition it descends from. Its mantra count, 5,839, stands against an attested total of roughly 5,977, and the divergence is recorded as open research rather than resolved.

Internal: agent non-independence and correlated error. The adversarial design assumes that an attacking agent fails differently from the agent it attacks. Where both are the same model family, that assumption is weak, and a shared blind spot would be invisible to the whole regime. We have no measurement of this. The one relevant datum is discouraging: single-model adjudication agreed with published human annotation on only 75 % of the 72 decidable cases where the two could be compared.

Internal: a single human decision-maker. The human arm of the workflow is one person, who is also the project’s author and the author of this paper. Owner rulings are recorded with their reasoning, which makes them auditable, but they are not independent of the project’s goals, and there is no second reader anywhere in the pipeline.

Internal: thresholds are chosen, not learned. The alignment floor of 0.62, the margin of 0.06, the near-parallel similarity cut, and the formula-length bounds are all set by hand. Some were set after inspecting their effects, which is a form of fitting to the corpus.

Internal: normalisation is lossy by construction. The lateral fold merges two phonemically distinct letters because two source conventions use the same code point for both (Section 3.4). This is documented and was measured, but it means a small number of comparisons are structurally incapable of distinguishing two words.

Conclusion: what the layer distribution does and does not show. The 0.17 % figure is a faithful measurement of what is *in* the graph. It is not a measurement of how much agents contributed to the project, which was very large, nor of how much they contributed to what the graph *says*, since an agent that finds and removes a wrong edge shapes the graph without appearing in it. We intend the figure to answer only the question of authorship of claims.

Conclusion: no absolute precision for the semantic layer. The reference set cannot decide aboutness or referent, reaches 3,072 of 20,210 mantras, and reaches no Sāmavedic mantra at all. We therefore report no precision figure for the semantic assertion layer as a whole, and we regard any such figure derived from model adjudication as unsupported.

External: generality. The method was developed on one corpus by one person over seventeen days. The corpus is unusually favourable in one respect — it is closed, small enough to enumerate, and equipped with a traditional metadata apparatus — and unusually hostile in another, in that its normalisation is genuinely contested. We discuss what we think transfers in Section 10.

Internal: two failing invariants, and what they mean for the rest. Two of the twenty live invariants fail (Section 8): 165,788 edges carry no grade rationale and 19,598 carry no attribution precision. The second is the more serious for this paper’s argument, because Section 4 presents attribution precision as a graph-wide axis and 3.8 % of edges do not carry it. Neither failure affects the layer distribution — the quality tier is present on every edge and the knowledge layer on 99.14 % — but a reader should treat any statement here about a *universal* property as holding for the properties we name as verified and not beyond them.

Internal: thresholds were selected on the corpus they are evaluated on. The near-parallel floor is justified by inspecting what a lower floor produces on this corpus; the alignment floor and margin are likewise hand-set. There is no held-out set anywhere in the project. Every precision figure in Section 8 is therefore optimistically biased by an amount we have not measured.

Internal: recall of the candidate generator is not zero-loss. The banding is described in the source as tuned so that every pair at 0.44 or better is proposed. For 32 bands of 3 rows the standard locality-sensitive-hashing curve puts the probability of proposing a pair at similarity 0.44 at roughly 0.94, not 1. We report the source’s claim as the source’s and do not adopt it: the layer’s recall below the floor is unmeasured.

Construct: a single decider with a stated commitment. The human arm of the workflow is one person, who is the project’s author, this paper’s author, and — as the acknowledgments record — someone with a devotional relationship to the subject matter. We raise this here and not only there, because it is a conflict of a different kind from the goal-alignment problem already noted: a reader is entitled to ask whether the owner rulings that decide scope and withholding are independent of a commitment to the material. We do not think the record shows them distorted — the rulings that matter most are refusals — but we cannot establish it from inside, and no second reader exists anywhere in the pipeline.

Provenance gaps. A registry statement and the projected graph can drift. We found one live instance while preparing this paper: the Sāmavedic work node’s completeness property still carries a pre-import sentence asserting that zero translations are released, where the registry file that is the source of truth was updated at the import to state the position precisely — that the 173 renderings are reused Ṛgvedic English on verified-identical Sanskrit and are counted in no Sāmavedic translation figure. The registry is right and the projection is stale. The general hazard is that prose-valued properties are not covered by the invariants that cover typed ones.

10 Discussion

10.1 What agents were actually good for

The measurement in Section 8.2 invites a deflationary reading: agents contributed 0.17 % of the graph’s edges, so they barely mattered. That reading is wrong, and the reason is worth stating precisely. Agents were the project’s instrument for four things that deterministic code could not do and a single human could not do at this scale.

Reading heterogeneous sources into a schema. Deciding that a particular TEI file has no editorial-alternative elements despite its header’s boilerplate, or that a translator’s preface commits him to a different recension, is reading. It is error-prone, it is cheap for a model, and it is checkable afterwards.

Generating candidates cheaply enough to be thrown away. The project’s most valuable candidate sets are the ones that were discarded. Discarding 49,179 role fillers is only a rational move if producing them was cheap; the same is true of the semantic-resemblance layer that never landed. Agentic generation changes the economics of refusal.

Cross-file reconciliation. Several of the most consequential defects — a registry claiming a closure the graph did not hold, a report citing a path whose file did not contain the digests described, a gate computing a set that made its own question unanswerable — are inconsistencies between artefacts that no single test covered. Finding them is a reading task over hundreds of files.

Adversarial probing with a stated attack. The Gate C case (Section 7.1) is the clearest instance. The attack — “if a withheld verse’s exact normalised text is released on a coreferent twin, the stated reason is false” — is a hypothesis about the project’s own explanations, and it is the kind of hypothesis an agent can be asked to generate and then required to test mechanically.

What agents were *not* used for is equally definite: they did not decide what was true. The promotion gate is a frozen set of trust classes, the candidate status is the only status a model may write, and the unlocked-predicate set is empty by construction.

10.2 What should stay deterministic

Three functions, on the evidence here, should never be delegated.

Identity. A model asked whether two verses are the same verse will answer, and its answer will be a similarity judgement wearing an identity claim's clothes. Identity in VedGraph is a pure function of declared coordinates, and the one place where a normalised label is an identity input is scoped to the candidate layer and named as such.

Promotion. The transition from candidate to accepted must be a property of the pipeline, not of the proposer, and it must be expressed as data so that it can be audited. Expressing it as a reviewer's convention would have failed: we observed a case where a completed review had simply never been applied, leaving 149 assertions live that a reviewer had not accepted, and the graph's count of reviewed edges at zero not because nothing had earned it but because the earning was never recorded.

Arithmetic over the graph. Every count in this paper is a query. The project's own Q38 failure (Section 8.6) shows why: a synthesised answer in which every figure was real and correctly cited was still misleading, because the comparative words around the figures — “most widely”, “largest” — asserted a ranking the cited rows did not contain. Numbers can be checked mechanically; superlatives had to be given their own rule.

10.3 Epistemic typing as an engineering discipline

The four-layer model is not novel as an idea — provenance vocabularies in the Semantic Web have expressed comparable distinctions for two decades — but three things about its use here seem worth transferring.

Type at the edge, not at the dataset. A dataset-level provenance statement cannot express that one predicate carries claims from three different kinds of evidence. VedGraph's attribution-precision axis exists because HAS_DEVATA is simultaneously a per-verse statement and an inherited label, and the split (Fig. 8) is roughly 1:3.

Give absence a type. The strongest design decision in the project is that a withheld row states a reason drawn from a closed class space, and that the count of rows with no disposition at all is gated at zero. This is what made the Gate C finding possible: you cannot falsify a reason that was never recorded.

Make the gate able to fail, and prove it. Two gates in this project were found to be incapable of failing, and a third's remediation shipped with ten mutant deletions verified to turn the tests red. A passing gate is evidence only if a failing version of the same gate has been demonstrated.

10.4 Generalisability

The Vedic corpus is favourable in ways that matter. It is closed and enumerable; it carries a traditional metadata apparatus that supplies genuine source-explicit attributions; and its recensional structure forces the scope question to be answered explicitly. A corpus that is open-ended, or that has no equivalent of the Anukramaṇī, would have a much thinner L1.

It is unfavourable in one way that may be more common than it looks: its normalisation is contested, so the comparison surface is a research object rather than a preprocessing step. Any corpus with competing transliteration conventions — which is most historical corpora outside the Latin alphabet — inherits the problem that a fold inside an identity digest is not free to change (Section 3.4).

We would expect the following to transfer: separating identity from normalisation; typing absence; expressing the promotion gate as data; requiring a negative control in any agreement measurement; and preserving failing gate runs under test. We would not expect the specific 0.17 % ratio to transfer, since it is a consequence of this corpus having a large deterministic layer available to it.

10.5 Relation to how such systems are usually built

Two contrasts are worth drawing explicitly. Retrieval-augmented pipelines typically treat provenance as a citation attached to an answer; here provenance is attached to the claim in the store, so a consumer can filter before an answer exists. And agentic extraction pipelines typically use a confidence score to decide what to keep; this project refuses that on the stated ground that the score is produced by the same process as the claim. The alternative it adopts — per-predicate unlocking gated on a measured precision against an external reference — has the property that nothing was ever unlocked, which is an honest outcome rather than a failure of the mechanism.

11 VedAnvaya as a Research Interface

The graph is exposed through a read-only API and a web interface called VedAnvaya. We treat it here as evidence for one narrow claim — that the evidence typing survives into use — rather than as a contribution in its own right.

The design constraint that matters is that typing is rendered *at the point of reading*, not collected on a methods page. Figure 9 shows the effect on a single Sāmavedic verse. Four distinct epistemic situations are visible simulta-

neously, each labelled in place: the svara marks are shown with the note that they are reproduced exactly as the source edition prints them, are a record of that edition’s notation rather than a reconstructed melody, and that nothing derives a sung realisation from them; the English is labelled a rendering *via* the verified Ṛgvedic parallel, with a scope note stating that it is not an independent translation of this corpus and is excluded from its translated count; the deity named inside the verse is marked PROBABLE rather than asserted; and the traditional ascription is marked “not yet modelled” rather than omitted.

The same discipline governs synthesis. The question-answering surface displays the retrieved evidence beside the prose, gives each cited item an “*what it does not establish*” affordance, and closes with a section headed “what this answer does not settle” that is populated by the mechanical auditors rather than by the model — in the case shown, a warning that the answer contains Sanskrit appearing nowhere in the retrieved evidence, and a note that the deity counts quoted include certain and probable referents and exclude ambiguous ones because a deity’s name in Vedic Sanskrit is also an ordinary noun.

Two behaviours are worth recording because they are unusual for an interface of this kind and both follow from the graph rather than from interface design. The question-answering layer declines 17 of 60 benchmark questions as insufficiently evidenced (Section 8.6). And a corpus page reports coverage as four populations rather than one percentage, so that the Sāmaveda’s zero independent English translations is legible next to its 173 reused renderings instead of being averaged into a single figure that would be true of neither.

We also report the interface’s most instructive failure, because it is a graph-level lesson. Restoring 30,266 semantic-assertion nodes — a correct restoration of real data — pushed the share of the exported public graph that is apparatus rather than entity to roughly 81 %, so that a visualisation of Indra’s neighbourhood at a fixed budget spent 21 of 40 slots on passages and evidence records and 6 on other deities. Nothing had become false; the graph had simply stopped being legible at that budget. The repair was per-class display ceilings, not a change to the data. Completeness and legibility are different objectives, and an interface over an evidence-typed graph has to be designed for both (Section 9, F6).

12 Limitations and Future Work

Section 9.3 treats threats to the validity of what we report. This section treats what the artefact does not yet do, and what we think should be done next.

The gap a single annotator cannot close. The decisive limitation is the absence of human annotation (Section 8.3). Everything downstream inherits it: no predicate can be unlocked, so no semantic assertion can be auto-accepted; no calibration curve can exist; and the only independent check on model adjudication we have — 75 % agreement with published human annotation on 72 decidable cases — is a warning rather than a measurement of the layer. The useful next step is small and specific: commission a stratified sample of a few hundred assertions annotated by two Vedicists with an adjudication protocol, report κ , and use it to decide whether any predicate clears the 95 % precision bar the policy already defines. The machinery to consume such a set exists and is currently idling against an empty scaffold.

Layer typing has a measured hole. 4,390 edges carry no knowledge layer, concentrated in two predicates (Section 8). This should be closed by extending the grading contract rather than by back-filling values, since the reason they are untyped is that the contract does not cover them.

Registry-to-projection drift is not gated. We found a live instance while writing this paper: a prose-valued scope property on a work node that the registry had corrected and the projection had not (Section 9.3). Typed properties are covered by invariants; prose ones are not. A cheap fix is to hash the registry’s prose fields into the projection and gate on equality.

The Sāmaveda is unscorable and partly unreachable. No external annotation reaches a single Sāmavedic mantra, so 11,380 enrichment edges on that corpus cannot be evaluated at all, and no Kauthuma Ārcika translation was found in nine searched avenues. The *gāna* corpus — larger than the Ārcika and the reason the Sāmaveda is a distinct Veda — is out of scope entirely and would need its own work identifier, its own sources and its own rights analysis.

The role layer is absent, not incomplete. Withdrawing 96 % of the role fillers left a predicate layer with no argument structure (Section 9, F2). Restoring it properly requires a dependency parse of the Ṛgveda that the project does not have; the 15,704 withdrawn rows are staged as a verification queue against that future parse rather than deleted.



[Ask](#)
[Vedas](#)
[Explore](#)
[Connections](#)
[Graph](#)
[Visualize](#)
[More ▾](#)



SVARA NOTATION

KAUTHUMA ĀRČIKA, SASVARA TEXT

ॐ ३ १ २ ३ १ २ ३ ३ ३ १ २ १ २ २
अग्न् आ याहि वीतये गृणानो हव्यदातये। नि होता सत्सि

३ ३ ३
बर्हिषि ॥ 1 ॥

[About these marks](#)

The svara marks above and below the syllables are reproduced exactly as the source edition prints them. They are a record of that edition's notation, not a reconstructed melody, and nothing here derives a sung realisation from them.

TRANSLATION

ENGLISH RENDERING VIA THE VERIFIED RIGVEDIC PARALLEL

Come, Agni, lauded, to the feast; come to the offering of the gifts. As Priest be seated on the grass.

Ralph T. H. Griffith, 1896 Ralph T. H. Griffith, The Hymns of the Rigveda, 2nd ed.
Rendered from RV 6.16.10

English rendering reused from the Rigvedic parallel of RV 6.16.10. The Sanskrit of the two passages is verified identical; this is not an independent translation of this verse's own corpus.

Scope note

At least one rendering here is reused from another corpus's published translation of verified-identical text. It is not an independent translation of the SV, and it is excluded from this corpus's translated count and from independent semantic evidence.

ASCRIBED IN THE APPARATUS

The traditional hymn heading, not a statement inside the verse.

Not yet modelled

NAMED INSIDE THIS VERSE

DEITY

Agni

PROBABLE

IDEAS, ACTS AND THINGS

CONCEPT

priest (hotr)

The officiant who invokes and offers, named by office rather than as an individual.

OBJECT

sacred grass (barhis)

The strewn grass on which the gods are invited to sit at the sacrifice.

CONNECTED PASSAGES

PARALLEL WORDING - TEXT REUSE

RV 6.16.10

Closely similar wording in both passages. One passage carries the wording of the other.

How attribution was recorded

THIS WORDING ALSO STANDS IN THE RIGVEDA

RV 6.16.10

Compare the texts side by side, with what each witness reads and how the connection was established

Show 13 connections in the graph →

Ask about this mantra

Figure 9. Evidence typing rendered where the reader is. **Top:** one Sāmavedic verse showing four epistemic situations at once – source-explicit svara marks with the note that they are a record of the edition’s notation and not a reconstructed melody; an English rendering labelled as reused via a verified Ṛgvedic parallel and excluded from this corpus’s translated count; a deity named inside the verse marked PROBABLE; and a traditional ascription marked “not yet modelled” rather than omitted. **Bottom:** the retrieved evidence beside a synthesised answer, with a per-item “what it does not establish” affordance and a closing section populated by the mechanical auditors rather than by the model. Captured from the running product against the graph census this paper reports.

Single-provider, single-sample evaluation. The downstream benchmark ran one model, one sample per question, and one adjudicator. Repeated sampling, a second provider and a second adjudicator would each be cheap and would each measure something the present figures cannot.

Agent independence is assumed, not measured. The adversarial regime’s value rests on attacking agents failing differently from the agents they attack, and we have no evidence for that beyond the defects the regime actually found. Running one gate under a deliberately different model family, and comparing findings, would be a direct test and is the experiment we would most like to see run.

Reproducibility stops short of the store. A fresh clone rebuilds the corpora and re-derives every identifier but cannot produce a certified graph (Section 8.7). Publishing a loadable dump is blocked by source rights rather than by engineering; publishing the *derived*, *project-original* layers separately — the grading, the gap registry, the staging evidence — may be possible and would let a third party re-run the gates.

No cost accounting. We report what agents produced but not what they cost in tokens, wall-clock or money, and a method whose argument is partly economic (Section 10) ought to report that. The run artefacts contain enough to reconstruct it for the semantic pilots; they do not for the construction work as a whole.

13 Conclusion

We set out to determine whether a large historical corpus can be turned into a provenance-aware knowledge graph with heavy LLM-agent involvement without model output becoming indistinguishable from what sources actually say. The answer this project supports is yes, but only because the constraint was implemented as mechanism rather than as policy: a model may write exactly one lifecycle status, the promotion gate is a frozen set of trust classes that does not contain the model’s, the predicate vocabulary is closed with its refusals named, and a cited token absent from the packet the model was shown is a decidable fabrication.

The resulting artefact is a graph of 165,737 nodes and 510,905 relationships over 20,210 canonical mantras in four named recensions, in which every edge states the epistemic layer that produced it, the tier that follows from it, the reach of the claim over its passage, and the textual surface the evidence was read off. Because the typing is stored rather than documented, the division of labour is measurable: 21.39 % of relationships are what a source states, 77.22 % are what a reproducible rule derives from that, 0.35 % are interpretive, and **0.17 % are model-extracted**.

That ratio is the paper’s main empirical result, and we want it read correctly. It does not say agents were unimportant — they were used continuously across seventeen days and 249 commits. It says that when the promotion path is closed, the agentic contribution flows into discovery, measurement, reconciliation and refutation instead of into assertion. The clearest instance is a gate built to attack the project’s own withholdings rather than its claims, which falsified the stated reasons of 39 withheld verses using the repository’s own released data, and changed no verse’s release status — only the recorded reason. A method that can find an error in its own explanations is doing something a method that only validates its outputs cannot.

The project’s failures support the same conclusion more strongly than its successes. It deleted 41,796 of its own relationships in one revision and retired 30,539 in the next; withdrew 96 % of its semantic role fillers rather than ship a measured 24.8 % error; refused an entire transcription corpus at an inter-reader agreement of 0.505; refused a semantic-resemblance layer after a re-implemented lexical control outperformed every candidate; and refused a release candidate while every gate was green, because completeness turned out to be a property of a registry that had no terminal state. Twice a refusal’s own decisive argument was subsequently found wrong and withdrawn in turn. Cheap generation makes cheap refusal possible, and refusal is where the epistemic work happens.

What none of this establishes is the accuracy of the semantic layer. Of 35,131 semantic assertions, zero have been reviewed by a human; the strongest review state in the graph is model adjudication; and on the only independent test available, single-model adjudication agreed with published human annotation on 75 % of decidable cases. The project says so in its own certification and on its public pages, and we have been careful to say it here in the same terms. An evidence-typed graph does not make model output true. It makes model output *visible*, and it lets a reader who does not trust it exclude it with a predicate — which, for scholarship over a corpus like this one, is the more useful property.

Responsible use

This project puts a generative model to work on a body of text that is scripture to living communities, and it publishes the result. Four commitments follow, and each is implemented rather than merely stated.

Hallucination is treated as decidable, not as a risk to be managed. The validator’s central check is whether a token the model cited was present in the packet the model was shown. A citation to text the model never saw is a fabricated citation, it is mechanically decidable, and the assertion is discarded. This does not detect a plausible-but-wrong reading of text that was shown, which is why model output remains a candidate regardless.

Uncertainty is preserved rather than resolved. Where the evidence does not decide, the graph says so in a typed way: 28,734 assertions carry withheld roles, 708 Sāmavedic verses carry a withheld notation disposition in one of three declared classes, 711 lexical tokens carry a linguistic non-resolution reason, and four benchmark questions are expected to be refused. The system is designed so that declining is a first-class outcome rather than a failure.

Missing evidence is never manufactured. Where a cue index would have needed interpolation between anchors to yield timings, the timings were refused, because filling the gaps means inventing a boundary. Where 466 metre rows could be resolved into a vocabulary that was itself contaminated, the resolution was computed and still not imported. We regard this as the single most important operational discipline in the project.

Interpretation is bounded, and the boundary is visible. The predicate vocabulary deliberately contains SYMBOLIZES, REPRESENTS, IS_GOD_OF, MEANS and CAUSES *so that a model proposing one is refused by name with a recorded reason*. Deity modelling is confined to what the Saṃhitā verses support: identifications that belong to later exegesis are declined individually and in writing. Musical notation is recorded as marks and never decoded into pitch, and a test scans the stored properties to enforce it (Section 7.1). None of this makes the graph neutral — every ontology encodes choices — but it makes the choices inspectable.

A residual risk we cannot design away is authority laundering: a confident interface over a typed graph can still be read as an oracle. The interface’s mitigation is to render the typing at the point of reading rather than in a methods page, and to prefer refusal; whether that is sufficient is not something we can establish from inside the project.

AI assistance disclosure

AI systems were used extensively and at every stage of the work reported here: software development, corpus inspection, candidate generation for the semantic and entity layers, query authoring, measurement, test and gate construction, adversarial review of the project’s own output, and the drafting and editing of this manuscript. The construction workflow described in Section 5 is not an incidental use of such tools; it is the object of study.

No AI system is an author. Responsibility for the research questions, the methodology, the interpretation of results, the claims made here and the decision to publish rests entirely with the human author. Where a model’s output entered the artefact it did so as a typed candidate subject to the controls described in Section 4.3, and the resulting population is reported rather than absorbed: 890 relationships and 2,459 nodes carry the model-extracted layer, and the graph can be queried to exclude them.

Model and runtime identities are recorded in the artefacts rather than treated as authorship. The project’s execution-runtime decision requires that the runtime that produced a payload be recorded and never assumed, and it explicitly classifies self-reported model identity as unattested: a receipt carries the runtime that wrote it, and provider-attested build metadata is marked unavailable because no provider supplies it. The reason given is directly relevant to this paper’s argument — an earlier pilot’s output had been recorded as model-generated when it was in fact produced by regular expressions, and the record was “provenance that looked like evidence and was not”. Specific providers and models used for each stage are listed in Appendix C for reproducibility, not as credit.

Data and code availability

The project source is public at <https://github.com/HimanshuMohanty-Git24/VedAnvaya> under the Poly-Form Noncommercial 1.0.0 licence. That licence covers project-original material only. It is *not* an OSI-approved open-source licence, and the project declines to describe itself as open source: the Open Source Definition’s sixth criterion forbids restricting a field of endeavour, and a noncommercial condition does exactly that.

What is in version control: the canonical corpus builds and their manifests, the source, rights and text-version registries, the identity and normalisation code, the projection and gate scripts, the staging evidence for each campaign, the decision records, the test suite, and — from the commit that adds them, which is necessarily later than the commit the measurements are frozen at — the materials for this paper including the figure and table generators and the fact freeze. At 6bf220c itself the paper directory is untracked, so a clone at exactly that commit does not contain the reproduction recipe in Appendix H; a clone at the paper commit does.

What is not: the loaded graph store itself; several source artefacts whose recorded rights do not permit redistribution, including the Atharvavedic text, which is held as a reference-only working corpus; the audio, which is

referenced rather than mirrored; and the generator of at least one checksum-sealed staging artefact, which is why a defect in it was corrected by an overlay rather than by editing the artefact in place.

One detail about the figures, because the paper’s own argument obliges it. [Figure 9](#) reproduces a screenshot in which three Atharvavedic verses appear in Sanskrit, drawn from a digital artefact the registry classifies reference-only. The verses themselves are ancient and the English beside them is public domain; we treat three verses shown to illustrate an interface as scholarly quotation, not as redistribution, and the corpus is not published with this paper.

Consequently **a fresh clone cannot reproduce a certified graph**, and we do not claim one-command reproducibility. What a fresh clone can do is rebuild the canonical corpora byte-identically from the pinned snapshots it is entitled to fetch, re-derive every passage identifier, and re-run the gates against a store it has loaded itself. Third-party texts, translations, annotations and recordings retain their own rights throughout; the registries record them per file, and the project’s own analysis is that a combined four-Veda release is not licensable as a single adapted work from the current sources.

Acknowledgments

This work depends entirely on scholarship and digitisation carried out by others: Aufrecht’s edition of the Ṛgveda and the GRETEL project’s TEI encoding of it, the VedaWeb platform and the Cologne linguistic annotation, the Treebank of Vedic Sanskrit, the Digital Corpus of Sanskrit, the digital Anukramaṇī of Akavarapu and Bhattacharya, Wikisource’s transcriptions, and the nineteenth- and twentieth-century translators — Griffith, and Whitney with Lanman — whose work remains the only complete English rendering reachable for parts of this corpus. Errors of method and of interpretation are the author’s alone.

The author personally offers this work with gratitude to Lord Viśvakarmā and to Indra, and regards himself as an instrument through which it was built. This is a personal devotional acknowledgment. It is not evidence, it is not a claim of any kind about the results reported above, and nothing in the scientific sections of this paper rests on it.

References

- V. S. D. S. Mahesh Akavarapu and Arnab Bhattacharya. Creation of a digital Rig Vedic index (Anukramaṇī) for computational linguistic tasks. In *Proceedings of the Computational Sanskrit and Digital Humanities Workshop at the 18th World Sanskrit Conference*, pages 89–96. Association for Computational Linguistics, 2023. URL <https://aclanthology.org/2023.wsc-csdh.6/>.
- Ron Artstein and Massimo Poesio. Survey article: Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4):555–596, 2008. doi: 10.1162/coli.07-034-R2.
- Theodor Aufrecht, editor. *Die Hymnen des Rigveda*. Adolph Marcus, Bonn, 2 edition, 1877. The printed edition behind the digital primary text used here, recorded in the project’s source-artefact registry.
- Maurice Bloomfield. *A Vedic Concordance*, volume 10 of *Harvard Oriental Series*. Harvard University Press, Cambridge, MA, 1906. An alphabetic index to every line of every stanza of the published Vedic literature, with an account of their variations across the Vedic books.
- Marco Böhler, Philip R. Burns, Martin Müller, Emily Franzini, and Greta Franzini. Towards a historical text re-use detection. In Chris Biemann and Alexander Mehler, editors, *Text Mining: From Ontology Learning to Automated Text Processing Applications*, pages 221–238. Springer, Cham, 2014. doi: 10.1007/978-3-319-12655-5_11.
- Neil Coffee, Jean-Pierre Koenig, Shakthi Poornima, Christopher W. Forstall, Roelant Ossewaarde, and Sarah L. Jacobson. The Tesserae project: Intertextual analysis of Latin poetry. *Literary and Linguistic Computing*, 28(2): 221–228, 2013. doi: 10.1093/lc/fqs033.
- Göttingen Register of Electronic Texts in Indian Languages. GRETEL. <https://gretel.sub.uni-goettingen.de/>, 2019. Niedersächsische Staats- und Universitätsbibliothek Göttingen. Source of the commit-pinned TEI encoding of Aufrecht’s Ṛgveda used here.
- Ralph T. H. Griffith. *The Hymns of the Atharvaveda*. E. J. Lazarus and Co., Benares, 1895.
- Ralph T. H. Griffith. *The Hymns of the Rigveda*. E. J. Lazarus and Co., Benares, 2 edition, 1896.
- Ralph T. H. Griffith. *The Texts of the White Yajurveda*. E. J. Lazarus and Co., Benares, 1899.

- Paul Groth, Andrew Gibson, and Jan Velterop. The anatomy of a nanopublication. *Information Services and Use*, 30(1–2):51–56, 2010. doi: 10.3233/ISU-2010-0613.
- Oliver Hellwig. DCS – the digital corpus of Sanskrit. <http://www.sanskrit-linguistics.org/dcs/index.php>, 2010–2021. Morphologically and lexically annotated Sanskrit corpus.
- Oliver Hellwig and Sebastian Nehrlich. Sanskrit word segmentation using character-level recurrent and convolutional neural networks. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2754–2763, Brussels, Belgium, 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1295.
- Oliver Hellwig, Salvatore Scarlata, Elia Ackermann, and Paul Widmer. The treebank of Vedic Sanskrit. In *Proceedings of the Twelfth Language Resources and Evaluation Conference (LREC)*, pages 5137–5146, Marseille, France, 2020. European Language Resources Association. URL <https://aclanthology.org/2020.lrec-1.632/>.
- Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutiérrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):71:1–71:37, 2021. doi: 10.1145/3447772.
- Wayne Howard. *Sāmavedic Chant*. Yale University Press, New Haven and London, 1977. ISBN 978-0-300-01956-8.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.01798>.
- Stephanie W. Jamison and Joel P. Brereton. *The Rigveda: The Earliest Religious Poetry of India*. Oxford University Press, Oxford and New York, 2014. ISBN 978-0-19-937018-4.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):248:1–248:38, 2023. doi: 10.1145/3571730.
- Börge Kiss, Daniel Kölligan, Francisco Mondaca, Claes Neufeind, Uta Reinöhl, and Patrick Sahle. It takes a village: Co-developing VedaWeb, a digital research platform for old Indo-Aryan texts. *Digital Humanities in the Nordic and Baltic Countries Publications*, 2(2):35–44, 2019. doi: 10.5617/dhnpub.11130.
- Timothy Lebo, Satya Sahoo, and Deborah McGuinness. PROV-O: The PROV ontology. W3C recommendation, W3C, April 2013. URL <https://www.w3.org/TR/2013/REC-prov-o-20130430/>. W3C Recommendation, 30 April 2013.
- Albert B. Lord. *The Singer of Tales*, volume 24 of *Harvard Studies in Comparative Literature*. Harvard University Press, Cambridge, MA, 1960.
- Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024. doi: 10.1109/TKDE.2024.3352100.
- Heiko Paulheim. Knowledge graph refinement: A survey of approaches and evaluation methods. *Semantic Web*, 8(3):489–508, 2017. doi: 10.3233/SW-160218.
- David A. Smith, Ryan Cordell, and Elizabeth Maddock Dillon. Infectious texts: Modeling text reuse in nineteenth-century newspapers. In *Proceedings of the 2013 IEEE International Conference on Big Data*, pages 86–94, 2013. doi: 10.1109/BigData.2013.6691675.
- William Dwight Whitney. *Atharva-Veda Samhitā: Translated with a Critical and Exegetical Commentary. Revised and Edited by Charles Rockwell Lanman*, volume 7–8 of *Harvard Oriental Series*. Harvard University Press, Cambridge, MA, 1905.
- Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi: 10.1038/sdata.2016.18.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.03629>.

A Ontology and Vocabulary Catalogue

A.1 Where the ontology lives

There is no single ontology file. Authority is composed across four modules, each owning a vocabulary, unioned at read time: a frozen semantic seal (node types and the closed predicate list), an enrichment layer (textual and structural predicates, plus the trust envelope), a domain layer (labels, predicates, endpoint signatures), and a grading layer (the per-edge grade and the attribution contract). A fifth vocabulary — the reader-facing one exposed by the API — is deliberately distinct from the writer’s (Section 4.2).

The composition function carries the method’s key statement, which we quote because it generalises beyond this project:

Critically, the graph is not an input. The previous scorecard gate computed `declared = ontology | everything in the graph` and then asked which graph types were missing from declared; nothing can be, so it reported 0 undeclared for a whole wave while eleven predicates went unclassified. **The data may not declare itself valid.**

A.2 The four typing axes

KnowledgeLayer

L1_SOURCE_EXPLICIT, L2_DETERMINISTIC_DERIVED, L3_LLM_EXTRACTED, L4_INTERPRETIVE_CLAIM. Documented as “never collapsed, never inferred from a score”.

QualityTier

TIER_A to TIER_D, a total bijection from the layer. One further transition: an L3 claim in state CANDIDATE is TIER_D, not TIER_C, because TIER_C means model-extracted evidence that survived review.

AttributionPrecision

PER_PASSAGE, CONTAINER_INHERITED, TEXTUAL_MENTION, NOT_AN_ATTRIBUTION. The fourth member is what makes a graph-wide universal-quantifier invariant over this property expressible at all.

Evidence surface

SANSKRIT, STRUCTURAL, SOURCE_METADATA, MIXED, TRANSLATION, SHARED_REGISTRY_ENTITIES. Stored under the property name `evidence_basis`, which collides with an unrelated derivation enumeration of the same name (Section 4.2).

Two legacy spellings of the layer field survive from an earlier pipeline generation (SOURCE_EXPLICIT and DETERMINISTIC_DERIVED without the L-prefix), on 6,236 and 8,214 edges respectively. The fact freeze reports both raw and folded distributions.

A.3 Lifecycle vocabularies

Six lifecycle vocabularies exist, deliberately not unified, each scoped to one layer. The two that matter for this paper are the graph-edge state (CANDIDATE/ACCEPTED/REJECTED, with the promotion gate encoded as a frozen set of trust classes) and the semantic-artifact status (CANDIDATE, AUTO_ACCEPTED, HUMAN_ACCEPTED, HUMAN_REJECTED, VALIDATION_REJECTED, SUPERSEDED, NEEDS_REVIEW), of which a model may write only the first.

Rejections carry one of twenty codes, one code per failure mode, including EVIDENCE_TOKEN_NOT_IN_PASSAGE, EVIDENCE_HASH_MISMATCH, PREDICATE_FORBIDDEN, PREDICATE_NOT_WHITELISTED, EXPLICITNESS_UNSUPPORTED_BY_EVIDENCE and DUPLICATE_ASSERTION.

Review verdicts applied to candidates are ACCEPT_MODEL_REVIEWED (promoted to TIER_C, review_state set to MODEL_ADJUDICATED and never to HUMAN_REVIEWED), REJECT (the edge is deleted, since a reader running the obvious query never sees the tier, and the row stays on disk so the deletion is reproducible), and AMBIGUOUS / NEEDS_MORE_EVIDENCE (kept at TIER_D and stamped).

There is no WITHDRAWN state. SUPERSEDED is the nearest.

A.4 Withholding mechanisms

WITHHELD is not a member of any enumeration. It is four separate typed mechanisms, each scoped to a layer: a reason string per unmapped sealed predicate, emitted into the load report alongside a count of predicates that are neither mapped nor explicitly withheld; a typed node property with a declared class space (Sāmavedic notation, three classes); typed non-resolution rows in the lexical layer (711 tokens, three linguistic reasons); and edge-type-keyed withholding for near-parallel pairs.

A.5 Node labels and relationship types

55 labels are in use and 81 relationship types, of a token table declaring 56 labels — `SourceArtifact` is declared and carries no nodes. The full distributions are in `supplementary/census_node_labels.csv` and `supplementary/census_relationship_types.csv`; the fourteen largest of each are in [Table 7](#).

A.6 Grades the bulk stamper must not touch

Twenty-one predicates are excluded from the generic grade sweep, each with a recorded reason. Three classes of reason recur. A predicate whose grade is the output of an independent review cannot be reproduced by any provenance signature — a post-load regrade once silently erased 587 promotions in one pass. A predicate whose grade is the grade of the node across the edge cannot be computed by a stamper that cannot see across an edge. And a predicate whose tier depends on which derivation produced the row must have both directions of its mirror written byte-identically, or a query gets a different tier depending on which way it walked.

B Corpus Identity Scheme

B.1 The URN grammars

Identity is `uuid5` over a canonical URN. One grammar per work:

Work	Mantra URN template
R̥gveda Śākala	urn:vedagraph:mantra:rigveda:shakala: mandala:{m}:sukta:{s}:mantra:{v}
Yajurveda vsm	urn:vedagraph:mantra:yajurveda: vajasaneyi-madhyandina:adhyaya:{a}:mantra:{v}
Atharvaveda Śaunaka	urn:vedagraph:mantra:atharvaveda:shaunaka: kanda:{k}:sukta:{s}:mantra:{v}
Sāmaveda Kauthuma	urn:vedagraph:mantra:samaveda:kauthuma:{collection} [:prapathaka:{p}][:ardha:{r}][:dasati:{d}]:verse:{v}

Note the Sāmavedic grammar names its collection rather than numbering it ([Section 3](#)): the first ordinal slot denotes different collections in different witnesses, and the blast radius of the rejected reading was measured at 1,280 of 1,875 verses before the decision was taken.

Keys are padded for lexical sorting (VG:RV:SAK:M01:S001:V001); URNs are unpadded and are the hash input. Only the URN may never be re-spelled.

B.2 Verification

Recomputing every identifier from its URN over all four canonical builds:

Build	Passages	Dup. keys	Dup. URNs	Dup. UUIDs	Mismatches
R̥gveda	11,590	0	0	0	0
Sāmaveda Ārcika	2,342	0	0	0	0
Vājasaneyi	2,015	0	0	0	0
Atharvaveda (working)	6,590	0	0	0	0
pooled	22,537	0	0	0	0

B.3 The comparison ladder

Six named comparison forms, from the identity function on stored text to the cross-script surface: `SOURCE_ORIGINAL`, `NFC`, `ACCENT_PRESERVING_NORMALIZED`, `ACCENT_STRIPPED_COMPARISON`, `SEARCH_NORMALIZED`, `CROSS_SCRIPT_COMPARISON`. Stored text is never replaced: `text_original` and `text_nfc` are separate fields, and `SOURCE_ORIGINAL` is the identity function, verified by test.

Accent stripping removes only Vedic tone marks (U+0951–U+0954, the combining members of the Vedic Extensions block, plus U+0300, U+0301, U+030D, U+0331, U+032D), and adjudicates ambiguous combining marks per base letter. One consequence is worth stating, because it bears on the flagship case study: the Sāmavedic svāra

marks are *not* in those ranges. Every one of the 25,542 marks on the 1,136 accented witnesses sits in Devanagari Extended (U+A8E1–U+A8F3 — combining Devanagari digits one, two and three, plus the combining letters), and zero are in U+0951–U+0954. The implementation is unaffected, because the Sāmavedic search surface is reached by transliteration rather than by this fold; the enumeration above is nonetheless not a description of the accents this corpus actually carries: on a vowel base a tone-set mark is an accent; on a consonant base carrying exactly one such mark, the mark is part of the letter (*ś* is *s* plus U+0301); on a consonant base carrying more than one, the extra is the tone.

B.4 Deliberate non-folds and one deliberate loss

Avagraha is a real orthographic sign and is not folded, at a measured cost of 46 of 116 residual unclassifiable Yajurvedic pairs. A private-use code point occurring 20 times in one witness is not folded, because reading it as an anusvāra from context is a transcription judgement rather than a normalisation; it is registered as a source defect instead. Conversely the whole lateral series folds onto one sentinel because U+1E37 denotes the vocalic *l* in one source convention and the retroflex *ḷ* in another. Splitting it was implemented, measured, and reverted.

Case is folded *before* the transcription table, not after: folding the other way left upper-case variants unmatched and made the search surface non-idempotent. Reordering was measured to change the surface for 0 of 21,936 canonical text records.

B.5 The referent-drift guard

Two digests per key: one over the bytes the source spells, one over a normalisation-independent comparison surface. Both are compared against the previous release’s digests for the same key, and a difference is adjudicated into UNCHANGED_REFERENT, INTENTIONAL_REFERENT_CORRECTION, REFERENT_DRIFT (blocks release), REMOVED_INVALID_PASSAGE or NEWLY_DISCOVERED_PASSAGE.

Text equality alone is insufficient. Of 1,844 released Sāmavedic keys only 1,651 have a distinct comparison digest: 192 equivalence classes covering 385 keys share one, and 174 of those classes share a byte-identical raw digest. The guard therefore requires three fields to agree — comparison digest, source locator and source verse marker — before calling two observations the same occurrence.

Because the comparison digest runs through the search surface, adding a rule to the transcription table changes the digest of already-released keys and the guard reports drift where none occurred. This was measured (one rule moves five Yajurvedic keys, others four and one more) and resolved by splitting the fold tables: a frozen table inside the digest, and a free table for comparison only.

C Agentic Workflow Details

C.1 No orchestration framework was used

We searched the project’s dependency manifests for agent-orchestration libraries (ReAct implementations, AutoGen, LangGraph, CrewAI, Reflexion and comparable packages) and found none. The runtime dependencies are a web framework, a graph driver, validation and transliteration libraries, and optional vendor SDKs for the language-model layer. Orchestration is the project’s own artefact protocol.

C.2 Model and runtime provenance

Runtime is recorded, never assumed. The runtime vocabulary has two members, both meaning “an agent runtime authored this payload and Python validated it”; neither is provider-attested, and provider build metadata is recorded as unavailable for both. A validator checks a receipt’s runtime against its run contract exactly as it checks model and reasoning settings, so a receipt cannot be silently imported into a run executed on a different runtime.

The rule exists because of a specific failure: earlier pilot output had been recorded as model-generated when it was in fact produced by regular expressions. The decision record calls this “provenance that looked like evidence and was not”, and notes that a wrong runtime string is the same defect in one field. A residual is recorded rather than fixed speculatively: one full-run module still pins an older model identity and will block a non-matching full-corpus run.

Models used, by stage, as recorded in the artefacts: the semantic-extraction pilots ran under two coding-agent runtimes across several versions, with the final 448-passage run recorded against a Claude Opus model; the theonym reference set is labelled MODEL_ADJUDICATED by the same; and the downstream question-answering benchmark was generated through an OpenRouter-hosted open model at temperature 0.1 with a 1,600-token output cap. These are recorded for reproducibility, not as credit (see the disclosure in the back matter).

C.3 The final closure sprint

Forty-three implementation-fixable gaps, partitioned across seven domain agents under the rule that every row has exactly one owner and none is unassigned:

Agent	Gaps	Domain
AGENT_1_FORMULA_CROSS_VEDA	3	formulae, cross-Veda reuse
AGENT_2_ENTITY_DEVATA_IDENTITY	12	entities and deity identity
AGENT_3_MORPHOLOGY_SEMANTICS	7	morphology, semantic assertions
AGENT_4_RITUAL	4	ritual and material culture
AGENT_5_QUALITY_REFERENCE	6	quality, reference sets
AGENT_6_PRODUCT_SURFACE	5	product-facing surfaces
AGENT_7_TRANSLATION_SAMAVEDA	6	translation, Sāmaveda
AGENT_8	0	audit of the other seven
lead	—	integration and rulings

Each domain agent produced measurements, row-level evidence and a `proposed_mutations.json`. None wrote to the graph. The auditing agent’s recorded database access is `READ_ONLY`. The lead produced an integration plan, four rulings and a receipt.

C.4 What the auditing agent found

Beyond the signature conflict described in [Section 5.4](#):

- A registry-consistency check that reported zero disagreements because it “only compared a graph measurement to a pre-declared integer; it never read the status field”. Four real disagreements surfaced once it did.
- Gate coverage over the registry: 53 entries gated, 32 ungated, and nine closures with no live measurement behind them.
- The exported public artefact under-declared its own inputs: 31 of 36 exported labels and 50 of 55 exported relationship types undeclared, hiding 32,044 node rows and 129,691 edges from the hash meant to cover them, with 4,016 public nodes dropped silently by the export.
- A search defect found by querying rather than by reading code: 5,123 of 5,839 search derivatives in one corpus carried a private-use sentinel, so the natural query returned 0 hits where the sentinel form returned 94.

C.5 The four lead rulings

R1 — cross-Veda text identity

Does an EXACT mapping confidence cover the key or the claim? Ruled: the key, and independently backed — the lead measured that all 216 target verses already carry a pre-existing parallel edge in the live graph and that none rests on a fold computed at staging time. Admitted as disclosed inherited analysis, under a standing constraint that these assertions may never be counted as Sāmavedic morphology on any surface.

R2 — role re-anchoring

See [Section 5.4](#) and [Fig. 4](#).

R3 — a corrected figure

The agent measured the live layer at 4,865 unreviewed assertions; its own staged delta takes the post-integration value to 35,139 (a prediction; the layer holds 35,131 today). The ruling also records a *trade*: closing one coverage gap multiplies another’s unreviewed population sevenfold and deepens a third.

R4 — a property-only correction

Does writing properties violate an owner decision that “the graph is not mutated”? Ruled no, on a reading of the decision’s own text: it forbids *importing* the unresolved objects, and the proposed change imports none — 0 nodes, 0 relationships, 466 property writes, no attribution created, no object resolved. Conditions attached, including that the census delta be verified as zero at integration.

C.6 The import protocol in full

Plan (hashed, enumerating nodes, edges, property writes and the expected census delta) → dry run (re-computed against the live graph, emitting GO or a reason) → owner GO against a specific plan hash → execute → readback

(rows sent against rows landed, census delta against the promise). Dry runs are regenerated whenever the working tree changes.

D Validation Gates and Invariants

D.1 Live invariants

Twenty invariants run against the loaded store, sixteen at failing severity. Each is phrased as the defect it detects rather than as the desired state, because each corresponds to a defect the project shipped at some point.

Two of them fail at the frozen commit: `edges_without_grade_basis` at 165,788 and `edges_without_attribution_precision` at 19,598, with `assertions_without_predicate_edge` warning at 2,193. See [Section 8](#) for how that correction reached this paper. Representative members:

Invariant	Contract
<code>ungraded_edges</code>	no edge without a quality tier
<code>edges_without_grade_basis</code>	no edge without a recorded grading rationale
<code>unlabelled_nodes</code>	no node with an empty label set
<code>product_node_reaching_diagnostic</code>	no public node reaches a diagnostic node
<code>works_without_scope</code>	no work without a scope statement
<code>works_without_excluded_corpora</code>	no work without a declared exclusion list
<code>edges_without_attribution_precision</code>	graph-wide, no null
<code>attribution_precision_outside_enum</code>	closed-world on the four values
<code>attribution_claimed_with_no_passage</code>	endpoint-level truth, checked independently “so that the table cannot certify itself”
<code>mention_edges_without_certainty</code>	every deity mention carries a referent certainty
<code>mention_edges_with_retired_value</code>	closed-world; a MERGE never forgets a superseded value
<code>predicates_with_multiple_run_ids</code>	else part of the graph is reproducible from no committed artefact
<code>passages_without_canonical_citation</code>	no passage a reader cannot cite
<code>rishi_family_without_evidence</code>	family is never assigned from name resemblance

D.2 Scorecard gates

Eleven integrity gates, each of which must read zero, with the process exiting non-zero otherwise: internal leakage, ungraded edges, nodes without a label, orphan entities, signature violations, undeclared relationship types, interpretive claims graded other than the weakest tier, claims pointing at passages, metrics pointing at passages, orphan public nodes, and labels falsely declared unpopulated.

Two are worth singling out. *Undeclared relationship types* was rewritten twice; both earlier versions were untestable and therefore untested, and both shipped unable to fail. The current implementation is a pure function of two arguments so that a test can prove it fails. *Orphan public nodes* was added late, because the earlier gate was scoped to one label family and read zero while 28 retired metre identities sat edgeless and public; its exemptions are per-label with a stated reason, on the principle that a numeric threshold would absorb the next real orphan.

D.3 The three-gate structure

Adopted across fourteen domains after four of four failed a third gate that did not previously exist.

Gate A — integrity

the artefact’s manifest checksums hold.

Gate B — conformance

rows are checked against the live graph for coverage, addressing, provenance and the central importability policy. The translation packet’s Gate B ran 38 checks over 2,254 rows at full coverage, with an explicit `checks_without_full_coverage` list that was empty.

Gate C — attack

implemented in a separate file, naming the fields it refuses to read, re-implementing the declared normalisation from prose rather than reusing the pipeline’s code, and carrying a negative control.

D.4 Readback

Every projection step returns rows *sent* alongside rows *landed*, and a step is complete only when the two agree and nothing was skipped. The design note is that a projection reporting only what it sent cannot tell a successful write from a silently unmatched one. The completeness flag was itself corrected once, after being always false: “a completeness flag that is always False is not a gate”.

D.5 Gates that could not fail

Three cases are recorded and all three are instructive.

One gate computed its set of declared relationship types as the union of the ontology *and the graph*, then asked which graph types were missing from it. Nothing can be. It reported zero undeclared types for a whole wave while eleven predicates went unclassified.

One readback printed `READBACK_CLEAN` having compared zero rows against zero rows.

One audit script opened with an assertion pinning the graph census. When the census moved, the assertion aborted the script — and because nothing wired the script to a gate, it failed by never running rather than by going red. It is now backstopped by a test that goes red instead.

E Evaluation Queries

Every figure in this paper is produced by `figures-src/fact_freeze.py` and its companions; the queries below are the ones whose results are quoted in the running text. All are `MATCH/RETURN`.

Graph scale.

```
MATCH (n) RETURN count(n)
MATCH ()-[r]->() RETURN count(r)
```

Epistemic layer distribution (the paper’s headline).

```
MATCH ()-[r]->()
RETURN r.knowledge_layer AS layer, count(*) AS n ORDER BY n DESC
```

Legacy unprefix spellings are folded in reporting; the freeze carries both.

Typing coverage.

```
MATCH ()-[r]->() WHERE r.quality_tier IS NULL RETURN count(r)
MATCH ()-[r]->() WHERE r.knowledge_layer IS NULL
RETURN type(r) AS t, count(*) AS n ORDER BY n DESC
```

Corpus composition.

```
MATCH (m:Mantra) RETURN m.work_id AS work, count(*) AS n
```

Attribution precision, the distinction that changes a count.

```
MATCH (p)-[r:HAS_DEVATA]->(d:Devata {display_label:'Indra'})
RETURN r.attribution_precision AS precision, count(*) AS n
```

Returns 2,214 container-inherited against 655 per-passage: a reader counting “Indra’s verses” from this predicate alone reports 2,869 where the source states 655.

Human review, the claim the paper most needs to be able to check.

```
MATCH ()-[r]->() WHERE r.provenance_class = 'HUMAN_REVIEWED' RETURN count(r)
MATCH (n) WHERE n.is_human_gold = true RETURN count(n)
MATCH ()-[r]->() WHERE r.review_state IS NOT NULL
RETURN r.review_state AS state, count(*) AS n ORDER BY n DESC
```

Return 0, 0, and {UNREVIEWED 7,548, MODEL_ADJUDICATED 613}.

Withholding in the semantic layer.

```
MATCH (a:SemanticAssertion)
RETURN a.roles_withheld AS withheld, a.three_slot_complete AS complete,
count(*) AS n ORDER BY n DESC
```

Sāmavedic notation disposition.

```
MATCH (m:Mantra {work_id:'VG:WORK:SV:KAU'})
RETURN m.samavedic_notation_state AS state,
m.samavedic_notation_withheld_class AS class, count(*) AS n
```

The invariant behind it — verses carrying no disposition at all — must read zero.

Typed translation coverage.

```
MATCH (m:Mantra)
RETURN m.translation_coverage_state AS state, m.work_id AS work, count(*) AS n
```

Cross-textual reuse by work pair.

```
MATCH (a:Mantra)-[r:REUSES_TEXT_FROM]->(b:Mantra)
RETURN a.work_id AS source, b.work_id AS target, r.method AS method, count(*) AS n
```

F Failure Case Ledger

[Table 8](#) summarises; this appendix records the artefact each case rests on, so that a reader with the repository can re-derive it. Paths are relative to the repository root.

F1 — 41,796 edges deleted, then 30,539 retired

docs/reports/VEDAGRAPH_KNOWLEDGE_MODEL_V2_FINAL_REPORT.md; docs/reports/KNOWLEDGE_MODEL_V3_FINAL_REPORT.md. The larger retirement was 21,246 concept edges resting on a single English keyword plus 9,000 deity mention edges proven to duplicate an existing layer fact-for-fact. Agreement between the concept layer and the mention layer moved from 55.3 % to 94.4 %.

F2 — 96 % of role fillers withdrawn

data/staging/semantic_roles/. Fillers 51,231 → 2,052; agent-plus-patient triples 5,219 → 341; 28,734 assertions stamped with a withholding reason; 15,704 rows staged as a verification queue. Repair was attempted first: widening the gate moved the wrong share from 24.8 % to 24.3 % at a cost of eighteen recall points, with per-role residue from 11.4 % (goal) to 31.2 % (location).

F3 — a corpus refused, then the refusal's reasoning corrected

docs/reports/ATHARVAVEDA_TRANSCRIPTION_QA.md and the commits that withdraw the monotone-trend claim. Agreement series: 0.00 (22 units), 0.26 (46), 0.485 (167), 0.505 (204), 0.542 (227), 0.522 (249), later 0.500 (272).

F4 — a refusal whose decisive argument was wrong

data/staging/communities/. The partition was refused, the refusal's reasoning overturned, and the refusal re-founded on a different measurable basis. The accompanying figure correction records both populations: tenfold imbalance over all 214 deities, 7.9-fold over the 157 eligible, peaking in different books.

F5 — release refused with every gate green

docs/reports/data-completeness/WAVE4_ADVERSARIAL_QA.md. Scorecard 11/11, dependency state current, all suites passing; verdict DATA_COMPLETENESS_NOT_COMPLETE, release candidate *no*. Registry: 84 of 85 entries open, no closure vocabulary, no field to hold one. The decisive finding was an entry recording a closure of 944 of 961 items against a graph holding exactly the pre-closure figure.

F6 — a correct restoration degraded the product

Restoring 30,266 nodes took the apparatus share of the public export to roughly 81 %; at a display budget of 40, the Indra neighbourhood spent 21 slots on passages and evidence records and 6 on other deities. Repaired with per-class display ceilings; the data were not changed.

F7 — 39 withheld reasons falsified

data/staging/samaveda_music_002_final/gate_c_run1.json, preserved as the failing run, with a test asserting it still records 39 defects. Class distribution moved 599/109/0 → 565/104/39, sum unchanged at 708; rows moving from withheld to released: 0; notation values invented: 0.

F8 — a layer refused before import

data/staging/semantic_resemblance/. Outcome VERIFIED_ZERO /REFUSED; 0 assertions imported. The re-implemented lexical control reached AUC 0.754 over 279 adjudicated pairs and nothing beat it. The prior run’s control figure did not reproduce (6 of 370 exact), and the carry-over argument was withdrawn entirely rather than reconciled.

F9 — 33 metre assertions withdrawn

The objects were unsegmented bracket fragments rather than metres. Three tests bounding the contaminated share of the target vocabulary disagreed — 17, 28, 39 — and the project’s recorded position is that the disagreement is the finding. All 466 related rows stay unresolved and non-importable.

F10 — four import defects caught by readback

data/staging/ wave artefacts: 5,930 edges over-written; 12,033 nodes left unreachable; 13,187 elements imported that the domain rules excluded; 102 curated display labels overwritten, 75 restored.

F.1 Two corrections the project made to its own corrections

Worth separating out, because they are the strongest evidence that the regime attacks its own reasoning and not only its data. In F3 a report described an agreement series as a monotone climb inside a paragraph written to correct an overgeneralisation; the series was not monotone and the claim was withdrawn with that observation attached. In F4 a refusal was upheld and its stated ground discarded, because the agent had imported the standard scholarly view as the graph’s model and then graded the graph defective for respecting the project’s own curation.

G Sources and Rights

G.1 The three-layer registry

Rights are recorded at three granularities that are allowed to disagree: the host (22 records), the individual file (53 artefacts, each with its own licence field), and the text layer (15 records). The governing statement is that rights attach to each source file, translation, transcription, image or recording, and that website accessibility does not grant reuse.

Thirteen numbered policy rules govern reconciliation. The load-bearing ones: on conflict the more restrictive statement controls; a permissive status is never inferred, only evidenced; absence of a licence field is not a grant; chain of title is followed to its root rather than to the nearest licence; a digitiser’s stamp is never a grant and a printed notice controls; suitability for one Veda does not transfer to another; and audio carries its own rights regardless of the text’s.

Status distribution at file level: UNKNOWN 15, PUBLIC_DOMAIN 10, APACHE_2_0 10, PERMISSION_REQUIRED 6, CC_BY_SA 6, REFERENCE_ONLY 3, CC_BY_NC_SA 2, CC_BY 1.

G.2 Text versions actually held

Text version	Role	Rights
GRETIL.RV.AUFRECHT	primary Sanskrit, Ṛgveda	CC_BY_NC_SA
VEDAWEB.AUFRECHT	parallel reading	CC_BY_NC_SA
VEDAWEB.EICHLER	parallel, accented Devanagari	CC_BY_NC_SA
VEDAWEB.VNH	metrically restored; <i>never</i> primary	CC_BY_NC_SA
VEDAWEB.LUBOTSKY	comparison only	CC_BY_NC_SA
VEDAWEB.PADAPATHA	declared, never ingested	CC_BY_NC_SA
VEDAWEB.ZURICH	linguistic annotation	CC_BY
WIKISOURCE_SA.SV.KAU.ARCIKA_MULA	primary Sanskrit, Sāmaveda	CC_BY_SA
GRETIL.SV.KAUTHUMA	primary; rejected witness	PERMISSION_REQUIRED
WIKISOURCE_SA.YV.VSM.ACCEntED	extracted from container	CC_BY_SA
WIKISOURCE_SA.YV.VSM.UNACCENtED	parallel reading	CC_BY_SA
GRETIL.AVS.SAUNAKA.ACCEntED	primary Sanskrit, Atharvaveda	REFERENCE_ONLY
GRETIL.AVS.SAUNAKA.UNACCENtED	parallel reading	REFERENCE_ONLY
VEDAGRAPH.AVS.ROTH_WHITNEY_1856.TRANSCRIPTION	the cancelled independent transcription	CC0
VEDAGRAPH.AVS.SEARCH_NORMALIZED	derived search surface	REFERENCE_ONLY

Three rows in that table are not data the project uses, and they are listed because their presence is the point: a declared-but-never-ingested Padapāṭha, a rejected Sāmavedic witness retained as the record of a defect family, and a cancelled transcription programme.

G.3 Printed editions behind the digital artefacts

The Ṛgvedic primary text descends from Aufrecht’s edition ([Aufrecht 1877](#)) through a data-entry, conversion and TEI chain that the artefact registry records step by step, with a pinned checksum and a publication date for the TEI encoding ([Göttingen Register of Electronic Texts in Indian Languages 2019](#)). That file’s header does not state a recension, so the Śākala identification is recorded as a reviewed structural inference rather than as a source statement.

Traditional Ṛgvedic metadata comes from a digital Anukramaṇī ([Akavarapu and Bhattacharya 2023](#)), commit-pinned, with a four-layer separation recorded explicitly: the traditional source is cited and not ingested; the digital transcription is ingested; modern editorial normalisation is recorded and never undone; and machine-classification labels in the same repository are never ingested at all.

English renderings are Griffith for the Ṛgveda, Yajurveda and Atharvaveda ([Griffith 1896; 1899; 1895](#)) and Whitney with Lanman for the Atharvaveda ([Whitney 1905](#)). Both are public domain; the project depends on the public-domain status of the *translation text* and deliberately not on the transcription layer carrying it, which is separately licensed.

G.4 The combined-release constraint

The Ṛgvedic primary text is CC BY-NC-SA, the Sāmavedic and Yajurvedic texts are CC BY-SA, and the Atharvavedic text is reference-only. The registry’s own analysis is that under CC BY-SA’s share-alike clause a combined four-Veda release is not licensable as a single adapted work from these sources. This is why the graph is buildable and queryable but not redistributable as one artefact ([Section 13](#)).

G.5 Snapshot provenance

An independent audit over the raw store verified 1,347 metadata records against their payloads: 1,347 hash-verified, 0 mismatches, 0 missing payloads, 0 orphans, 0 incomplete provenance records and 0 partial-download residue, across roughly 249 MB. Eleven sidecar fields are required per snapshot, including retrieval URL, retrieval timestamp, HTTP status, request headers and parser-independent metadata. A repository URL without a commit pin raises.

The audit states its own limits, and we adopt them: hash verification proves a snapshot is unaltered since retrieval and says nothing about whether retrieval was permitted, nor about whether the digest was pinned against the right artefact. Integrity and scope are independent properties.

H Reproducibility

H.1 What reproduces, measured

- **The canonical corpora rebuild byte-identically** from their pinned snapshots.
- **Every passage identifier re-derives** from its URN: 22,537 recomputed, zero mismatches ([Appendix B](#)).
- **Public identifiers agree across independent routes.** An audit asserts that the API’s identifier function and the offline export agree row for row, that no two public nodes share an identifier, that the passage and assertion identifier spaces are disjoint, and that two independent source builds produce a key set identical to the live graph’s.
- **Reproducibility was measured by re-running** rather than asserted, during the completeness campaign.
- **A run-identity invariant** fails the build if any predicate carries more than one run identifier, on the ground that otherwise part of the graph is reproducible from no committed artefact.

H.2 What does not reproduce from a clean clone

A fresh clone **cannot** produce a certified graph. Four independent reasons:

1. The loaded store is not in version control and is not redistributable as a dump, for the licensing reason in [Appendix G](#).

2. Several source artefacts cannot be re-fetched by a third party under their recorded rights; the Atharvavedic text in particular is held as a reference-only working corpus.
3. At least one checksum-sealed staging artefact’s *generator* is absent from the repository, which is why a defect in it had to be corrected by an overlay rather than by regeneration.
4. The audio layer is referenced rather than mirrored.

H.3 Rebuilding the paper’s numbers

The manuscript’s numeric content is a projection of one file. With the repository and a loaded store:

```
python figures-src/fact_freeze.py
python figures-src/graph_census.py
python figures-src/make_tables.py
python figures-src/fig_corpus_and_epistemic.py
python figures-src/fig_crossveda.py
python figures-src/fig_samaveda.py
python figures-src/fig_indra.py
tectonic -X compile main.tex
```

The first, second and seventh require the graph; the rest require only the freeze. The freeze records the commit, the branch and whether the tree was clean when it was taken, so a reviewer can tell whether a number in the text belongs to the state the paper claims for it. It is worth saying what it records here: **the tree was not clean**. `git_tree_clean` is *false* in the shipped freeze, because the paper’s own directory was untracked at the moment of measurement. We believe that is the whole of the delta — `git status --porcelain` reported only `?? paper/` — but the freeze cannot establish that on its own, and a measurement taken from a dirty tree is a measurement taken from a dirty tree.

H.4 Determinism hazards in the paper’s own tooling

Two, both found and fixed while preparing this manuscript, and recorded because they are exactly the kind of thing that makes a “reproducible” figure irreproducible. The subgraph generator originally derived diagram node identifiers from Python’s built-in hash, which is salted per process; it now uses positional counters. And a thousands-separator substitution was applied to a whole formatted string rather than to the number inside it, which silently rewrote a coordinate. Neither would have changed a reported figure; both would have changed the output between runs.